Unused Mantissa Bits on Power-Capped GPUs: Mechanism, Prediction and Limits on NVIDIA GH200 and AMD MI350X

Koying Chen
Department of Computer Science, National Tsing Hua University, Hsinchu, Taiwan

***Abstract*—When a GPU runs at its power limit, the energy of every operation sets the clock. Zeroing the low-order mantissa bits an application does not need, in software and on the original FP64, BF16 or FP8 units, lowers that energy; we measure what this buys on an NVIDIA GH200 cluster and two 8-GPU AMD MI350X nodes. Filling the discarded bits with ones instead of zeros, which stops them toggling without making them zero, retains 65–97% of the saving in matrix engines, vector units and memory copies, evidence that switching activity is the main contributor. Given the native program's performance–cap curve, a model with the measured idle power and two locked-clock measurements (or one capped run) predicts the truncated program's curve within 1.5 (GH200) and 2.1 (MI350X) percentage points on average, and within 2.9 points for the 27–36 FP64 bits applications keep. MI350X is power-bound at its stock 1000 W limit, so in-program truncation pays at default settings: FP64 HPL keeping 36 significand bits is 4.6–5.1% faster on two nodes, HPL-MxP keeping 3 bits on a diagonally dominant test matrix 4.9–5.1% (7.6 to 8.0 PFLOP/s), a Gaussian-process likelihood 7.6% and DF-MP2 10.2%, and 6–20% at 500 W, with every residual check passed and MP2 energies within 2.2×10^{-8} E_h. INT8 emulation of FP64 is faster for long inner dimensions at generous caps (36% above the best native HPL), but a tighter cap moves the crossover toward truncation; emulation also needs workspace that limits problem size, and fails silently on data with wide per-row exponent spans. Safe bit counts are platform-dependent, and chassis fan feedback confounds interleaved measurements unless temperature is modeled.**

***Index Terms*—power capping, energy efficiency, floating-point precision, mantissa truncation, GPU, mixed precision, HPL, emulation**

I. Introduction

Power is becoming the ceiling on delivered performance. Data centers cap rack power, student cluster competitions cap the power of a whole cluster, and accelerators ship with board or package limits that their own workloads reach: the AMD Instinct MI350X holds FP64, BF16 and FP8 matrix multiplication at its default 1000 W limit 82–100% of the time in our measurements. Under a limit, the power manager lowers the clock until the device fits; the speed of a program then depends on how much energy each of its operations consumes.

That GPU power depends on input data is known. Bhalachandra et al. measured 67% more power for DGEMM on random than on constant operands on A100 [1]; Gregersen et al. varied the distribution and bit patterns of GEMM inputs and found that zeroing the least significant bits lowers power [2]; He showed an all-zero matrix multiplication on A100 running about 15% faster than a random one, with a gap that widens as the power limit is lowered [3]; Antepara et al. measured 563 W for a zero-input DGEMM on GH200 versus 898 W for random input [4]; and the TensorRT documentation warns that all-zero inputs draw less power because fewer DRAM and L2 bits toggle [5]. In hardware, Tong et al. narrowed floating-point multipliers [6], Pool et al. selected precision for graphics shaders [7], [8], Guenther et al. masked mantissas at the inputs of arithmetic units [9], and an AMD patent adds a hardware zeroing module to shaders [10].

These works either demonstrate the effect with special inputs such as all-zero or constant data, on one vendor, or require new hardware or simulated energy. None answers the questions a practitioner faces. Does software truncation of the low-order mantissa bits of *real* data buy performance on production GPUs of different vendors, and through which mechanism? Can the gain be predicted before running the application under every cap? How much accuracy does it cost? And where does it still pay off against format changes such as FP8 or INT8 emulation of FP64 [11], [12], whose own speed also depends on the power limit? To our knowledge this is the first study that truncates part of the mantissa of real FP64/BF16/FP8 operands in software on production GPUs of two vendors and measures, on benchmarks and real HPC applications checked by their own correctness criteria, how much throughput the saved switching power buys under power caps. Our contributions are:

1) **Evidence on the mechanism, across vendors, units and clocks.** Filling the discarded bits with ones or a fixed pattern instead of zeros stops them toggling without making them zero. On GH200 and MI350X, in matrix engines, FP64 vector units and memory copies, at three fixed clocks on MI350X, such fills retain 65–97% (ones) and 41–92% (fixed pattern) of the zero fill's benefit (Section III): switching activity is the main contributor, although the shares are not an exact decomposition. On MI350X, truncation also raises the effective clock at a fixed target clock, a data-dependent clock reduction that no throttle counter reports.
2) **A predictive model.** If truncation only changes the switched capacitance, the truncated program under cap P behaves like the native program under $P_0+(P-P_0)/R$. Given the native curve, the measured idle power as P_0 and two locked-clock measurements (MI350X) or one

capped run (GH200), it predicts 42 cap–format points per platform with mean absolute errors of 1.5 and 2.1 percentage points (largest 4.6 and 10.4; at most 2.9 for 27–36 kept FP64 bits; Section IV).

3) **Gains on real applications on two platforms.** We express the benefit as the extra cap the unmodified program needs to catch up, from GEMM to HPL, HPL-MxP, Quantum ESPRESSO, DF-MP2 and a wide-range, power-bound Gaussian-process likelihood, with paired 95% intervals, replicated on a second node set on each platform, and check residuals and energies with each code's own criterion (Section VII). We separate deployable results, measured against the unmodified program with truncation inside it, from upper bounds obtained through an interposer. On MI350X the gains appear at the default limit; on GH200 mainly under lowered caps.
4) **The boundary with format changes.** Against INT8 emulation of FP64 (cuBLAS, GEMMul8) and FP32/3xTF32, and FP8 for AI, the crossover moves with matrix shape, power cap, memory headroom, accumulation precision and exponent span; a tighter cap favors truncation on both platforms, and on data with a wide per-row exponent span fixed-width emulation can silently return wrong results, a risk that a cheap measurement of per-row spans flags beforehand (Sections VIII–IX).
5) **Measurement lessons.** Hardware energy accumulators remove the sampling bias of driver power readings; chassis fans that react to the previous run change a power-capped GPU's speed by up to 4.5% through its temperature, which interleaving alone does not remove, so we model temperature explicitly; and safe bit counts must be validated on the target platform (Sections VI and VII-B).

Compared with the closest work: Gregersen et al. observed input-dependent power and proposed the toggling hypothesis but avoided throttling and studied only GEMM; He swept the A100 power limit, and Antepara et al. measured GH200, but both contrasted all-zero with random inputs without truncating real data or evaluating accuracy. We quantify the toggling hypothesis on two vendors' module and package power, turn it into a model that predicts the gain under a cap, show that the data-movement part is consistent with DRAM energy studies [13], [14], and quantify speed and accuracy on real applications under caps. Uchino et al. evaluated the speed and energy efficiency of INT8-emulated GEMM [12]; we compare it with truncation at the same cap and comparable accuracy on two platforms and find that the winner depends on matrix shape and cap.

II. Background and Related Work

Input-dependent GPU power. Bhalachandra et al. argue that power budgets and benchmarks must account for input entropy [1]. Gregersen et al. varied value distribution, bit similarity, ordering and sparsity of CUTLASS GEMM inputs on A100, H100 and V100 and saw power differ by up to almost 40%, with zeroed low bits lowering power [2]; they chose sizes that do not throttle, so performance under a cap was not measured. He reported 295 versus 257 TFLOP/s for all-zero versus random 8192^3 GEMMs on A100 and, lowering the power limit from 330 to 100 W, saw the relative speedup of such predictable inputs grow from about 1.2× to about 2.3× at 150 W before it fell at 100 W [3]. Antepara et al. measured zero-input DGEMM on GH200 at 563 W and 65.4 TFLOP/s against 898 W and 57.8 TFLOP/s for random input [4]; their GH200 roofline study found no significant difference in vector FP64 performance between zero and random inputs [15], consistent with our finding that when the GPU is not power-bound the difference shows only in power. Ujeniya et al. note that constant and random data can differ by about 100 W on H100/H200 but leave it out of scope [16]. Lucas and Juurlink built a data-dependent ALU power model for GTX 580 whose main term is the number of toggles, with the number of ones a secondary term [17]. All of these use all-zero, constant or special bit patterns on NVIDIA hardware; none truncates part of the mantissa of real data, evaluates accuracy or applications, or compares vendors.

Power-constrained HPC. Rountree et al. took a first look at performance under a hardware-enforced power bound and found that manufacturing variation makes nominally identical processors differ by about 6% in execution time under the same bound [18]; Inadomi et al. measured up to 64% variation in per-rank DGEMM performance under a module power cap on a large system and allocated power by variability [19]. Patki et al. overprovision nodes under a fixed system power budget and enforce per-node bounds [20], and Zeus tunes the GPU power limit together with the batch size to cut DNN training energy [21]. These works decide how power is distributed; we lower the energy each operation needs, so the two compose. The 13–17% node-to-node clock spread we see on GH200 at equal caps, and the 860–1200 MHz spread among the eight GPUs of one MI350X node at 800 W (Section VI), are consistent with their observations.

Hardware mantissa masking and data-movement energy. Mantissa masking itself is not new. Tong et al. showed that many programs tolerate halving the mantissa and that a variable-width multiplier saves up to 66% energy per operation [6]. Pool et al. estimated 23–36% energy savings in the vertex stage of mobile GPUs [7] and up to 71% in pixel-shader arithmetic (10–20% of GPU energy) [8]; Angerd et al. automated precision reduction in graphics and designed a register file that packs arbitrary precisions [22]; Guenther et al. masked mantissas at functional-unit inputs of an LTE baseband processor for up to 23% energy per task [9]; an AMD patent zeroes mantissa bits in GPU shaders according to a precision flag [10]; and Jahadi and Atoofian exploited the zero bias of register bits and dropped low-order bits of CNN values to cut tensor-core register-file leakage by 77.3% in simulation [23]. rpe and chop use the same bit operations to emulate low precision but do not measure power [24], [25]. DRAM and HBM energy depend on data toggling rates [13], [14], [26], and compression can reduce bytes but increase toggles [27].

We differ in being pure software, changing neither hardware nor data format, and in measuring clocks and performance under caps on production GPUs with validated application accuracy.

Precision tuning, mixed precision and power caps. Tools such as Precimonious lower variable precision for speed [28], and mixed-precision storage reduces bytes per value [29]; mantissa zeroing changes neither type nor bytes and composes with both. Transprecision computing argues for controlling precision at fine grain in space and time [30]. Closest in form, Grützmacher et al. store FP64 values as mantissa segments that keep the full exponent and read only the leading segments, speeding up memory-bound PageRank by 10–30% [31]; there the truncation saves bandwidth, here switching power. Low-precision LU with iterative refinement recovers FP64 accuracy [32] and underlies HPL-MxP [33]. Klöwer et al. estimate from the data itself how many mantissa bits carry real information and round the rest to nearest to compress atmospheric data [34]; the same analysis could estimate in advance how many bits GEMM operands need, here for power rather than storage. GH200 shares module power between CPU and GPU and lowers the GPU clock near the cap [35]; Yang et al. found that nvidia-smi samples only about 25% of runtime on A100/H100 [36], which we mitigate on GH200 with long steady windows, pairing and energy integrated from 1-s averaged power, and avoid on MI350X by reading a hardware energy accumulator (Section VI).

Emulating FP64 on low-precision units. The Ozaki scheme [37] splits a matrix product into a sum of products of lower-precision slices that are computed exactly; Ootomo et al. run it on integer matrix units [38], and Ozaki scheme II uses modular arithmetic [11] and is implemented in GEMMul8 for CUDA and HIP, with performance and power evaluated by Uchino et al. [12]. cuBLAS ships an Ozaki-type fixed-point emulation that scales each row of A and column of B by a shared factor, so the dynamic range within a row is encoded in the mantissa; its automatic mode picks the slice count from the exponent span and dispatches to native FP64 when more than a library-defined maximum of bits would be needed [39], [40]. The benefit of emulation grows with the ratio of INT8 to FP64 throughput, which is about twice as high on MI350X as on GH200 (Section VIII-B). These change the arithmetic format; zeroing keeps the original FP64 (or BF16, FP8) units and lowers input entropy, so the two can be complementary.

Mixed precision in geostatistics. ExaGeoStat evaluates Gaussian-process likelihoods with an FP64 tile Cholesky [41]. Its mixed-precision successors order the locations along a space-filling curve so that strong correlations sit near the diagonal, keep diagonal tiles in FP64 and convert tiles chosen by their distance from the diagonal or their norm to FP32, FP16 or FP8: 1.6× on average with FP64/FP32 [42], up to 2.64× with FP64/FP32/FP16 [43], up to 12× when combined with tile low-rank compression [44], and 3× over FP64 on a GH200 with four precisions [45]; the same tile method drove a climate emulator on GH200 systems [46]. Cao et al. also measured the power and energy of such factorizations on single V100, A100 and H100 GPUs [47], without caps or a clock analysis. Uniform single precision is insufficient for these likelihoods [43], as we also find. All of these convert whole tiles to another format; we keep FP64 and truncate operand mantissas, and the two compose (Section VIII-E). Olivares-Amaya et al. similarly ran RI-MP2 with single- and mixed-precision GEMMs on GPUs [48].

Convergence-driven precision. Woo et al. switch LOBPCG between FP32, FP64 and mixed precision by convergence state for up to 8.5× speedup on GPUs [49]; Huang, Shao and Hammond choose the number of INT8 emulation slices in GPU4PySCF from the relative SCF energy change, keeping errors below 10^{-7} hartree (E_h; $1\,E_h = 2\,\mathrm{Ry} = 627.5\,\mathrm{kcal/mol}$) [50]; Dawson et al. found with MPFR that about 37 significant bits suffice for density matrix purification [51]. All of these gain speed by moving to a higher-throughput format or unit, and none studies power. We stay on FP64 units, adjust the kept bits one at a time from the Davidson threshold of a plane-wave code, and gain mainly from clock headroom under a cap (on one of our two test systems also from fewer Davidson iterations; Section VII-D).

FP8 inference and its failures. FP8 training keeps high-precision master weights with per-tensor scaling, E4M3 for weights and activations and E5M2 for gradients [52]; Gupta et al. showed that rounding decides the fate of low-precision training [53]. The clearest FP8 failure is attention: FlashAttention-3's FP8 mode drops the VQA-a video-quality score of CogVideoX from 70.2 to 6.5 and of HunyuanVideo from 82.5 to 4.4, and 262k-context retrieval from 7.0 to 0.4, which SageAttention2 repairs inside FP8 by smoothing outliers and using a fixed scale for softmax probabilities [54]. MLPerf Inference closed division allows any pure, reproducible mathematical quantization but forbids pruning non-zero weights [55]; mantissa zeroing is the former.

III. Why Fewer Mantissa Bits Save Power

CMOS dynamic power is roughly $\alpha_{\mathrm{sw}} C V^2 f$, where the switching activity α_{sw} depends on the data. Under a cap, the power manager lowers f (and V) until the device fits, so a lower α_{sw} turns directly into higher clock and throughput; below the cap it appears as lower power at the same clock. We see both on both platforms. On GH200 at the default 1000 W module cap, the native FP64 GEMM is still capped 95% of the time and runs at 1822 MHz, whereas keeping 0 bits it runs at the maximum 1980 MHz while module power falls from 984 to 707 W; under a 700 W cap its SM clock is 1502 MHz native, 1668 MHz with 27 kept bits and 1965 MHz with 0 kept bits, all at about 694 W. On MI350X at its default 1000 W package power limit (PPT), the native FP64 GEMM is held at the limit 82% of the time at 1890 MHz; with 0 kept bits it never reaches the limit (785 W) and runs at 2183 MHz, 17% faster. Native BF16 and FP8 GEMMs are held at the limit 99.8 and 100% of the time, so every bit they stop toggling is turned into clock (Table I).

Toggling or zero? Zeroing changes two things at once: the low bits stop varying from element to element, and

TABLE I

Share of the zero-fill benefit (%) that a fill with ones (first number) or a fixed nonzero pattern (second) retains. GH200: module power at 1000 W or throughput at 500 W (Section III); MI350X: energy per operation at a locked clock of 900, 1300 or 1700 MHz (GEMMs; ranges over the clocks), throughput at 600 W or package power at 1000 W (other kernels). Last column: benefit of the zero fill on MI350X (E/op: energy per operation). On GH200 the zero fill gives 707 versus 984 W (FP64, 0 bits), +14.5% (FP64, 20 bits, 500 W), +28% (BF16, 500 W), 553 versus 862 W (FMA loop) and −18% power (copy).

Kernel, kept bits	GH200	MI350X	MI350X zero fill
FP64 GEMM, 0	76/68	73–75/69	−20 to −29% E/op
FP64 GEMM, 20 (G), 27 (M)	87/82	81–83/73–74	−7 to −10% E/op
BF16 GEMM, 0	92/92	77–78/68–72	−13 to −15% E/op
FP8 GEMM, 0	—	65–66/61	−9 to −10% E/op
FP64 FMA loop, 0	87/62	74/63	+33% speed
Memory copy, 0	97/86	83/41	−9% power

their value becomes zero. To separate the two we fill the discarded bits with all ones, or with a fixed nonzero pattern shared by all elements; neither toggles, but neither is zero. On GH200 at 1000 W the fills with 0 kept bits run at the maximum 1980 MHz (with 20 kept bits at 1963–1971 MHz, partly capped), so we compare module power; since the cap holds the native GEMM at 984 W, the share we attribute to "no toggling" is conservative. At 500 W every configuration is capped 100% of the time, so we compare throughput (one node, two runs each). On MI350X we can do better: we lock the maximum graphics clock at 900, 1300 or 1700 MHz, below the frequency any configuration reaches at 1000 W, so that no configuration is power-limited (PPT residency 0–0.6%) and the energy per operation compares switching activity directly (8 GPUs × ABBA per setting).

In these measurements merely stopping the low bits from toggling captures most of the benefit, in matrix engines, FP64 vector units and memory copies of both vendors (Table I; ones: 65–97%, fixed pattern: 41–92%): quantitative support for the toggling hypothesis [2] beyond one vendor or one unit. The shares are not an exact causal decomposition. A fill of ones or a pattern also changes the partial products, alignment and accumulation inside the units, and some shares compare power while others compare throughput under a cap, which need not add up. What the controls establish is that most of the saving survives when the discarded bits are non-zero but constant, so switching activity, not the value zero, is the main contributor. On MI350X the shares barely change with the clock (FP64 with 27 kept bits: 83, 81 and 82% at 900, 1300 and 1700 MHz; with 0 kept bits 74, 75 and 73%), while the total saving grows with it (FP64, 0 kept bits: 20, 24 and 29% less energy per operation), as expected when the switched-capacitance term CV^2f grows relative to static power. Zeros save somewhat more than ones or a fixed pattern, and the pattern saves least on the MI350X memory copy (41%). With 20–27 kept bits this matches multiplier structure: partial products are $a_i b_j$, so low bits of a fixed at one still form partial products that toggle with the kept bits of the other operand, whereas zeros keep that whole slice at zero. With 0 kept bits both mantissas are constant, so the partial products do not toggle under any fill, yet zeros still save more than ones (GH200: 707 versus 774 W); this may come from simpler alignment and accumulation of products that are exact powers of two, which we have not verified. We have no gate-level measurement, so both are inferences consistent with the data. The result also matches the GTX 580 ALU model of Lucas and Juurlink, in which toggles dominate and the number of ones in constant inputs is secondary [17].

Not just matrix engines. In an FP64 multiply-add loop without matrix units (8×8 outer product per thread), keeping 0 bits lowers GH200 module power at 1000 W from 862 to 553 W and makes MI350X 16% faster at 1000 W and 33% faster at 600 W. In pure memory traffic (a STREAM-like copy) GH200 module power falls by 18% at 1000 W, and on MI350X, where even a copy is power-bound at 600 W, the copy runs 28% faster. Fewer toggles on data wires and in arithmetic units are therefore the main cause, and the effect also exists for bandwidth-bound code; the data-movement part is consistent with DRAM energy that depends on toggling rates [13], [14].

A second, invisible clock effect on MI350X. With the clock locked and no configuration power-limited, the effective graphics clock still depends on the data. Locked at 1300 MHz, the native FP64 GEMM runs steadily at 1171–1179 MHz (5th–95th percentile over 8 GPUs), the same GEMM with 0 kept bits at 1235–1240 MHz, native BF16 at 1088–1100 MHz and BF16 with 0 kept bits at 1132–1142 MHz, at 59–63 °C junction temperature. The power and thermal residency counters read zero, and the firmware's per-XCD "clock below host limit" counters stay at zero even when the power limit is active, so this reduction is reported by none of the counters available to us. At 1000 W without a lock, FP64 with 0 kept bits likewise runs at 2183 rather than 2200 MHz with no power limiting. A circuit-level mechanism that stretches the clock when the supply droops under high switching current would explain these observations; we have no direct measurement of it. Whatever its cause, truncation relieves it too, and the model of Section IV must include it.

IV. A Model of Truncation Gains Under a Power Cap

Let a GPU under cap P settle at a target clock f with $P = P_0 + c\,g(f)$, where P_0 is the power that does not scale with switching (static and uncore), $g(f) = V(f)^2 f$ is the voltage–frequency curve of the device, shared by all programs, and c is the program's effective switched capacitance. Truncation changes only c. Let the effective clock be sf, where $s \le 1$ is the clock reduction of Section III (MI350X) and performance is proportional to the effective clock. Write $\sigma = s_r/s_n$ for the ratio of effective clocks of the truncated (r) and native (n) programs and $R = (c_r s_r)/(c_n s_n)$ for the ratio of their dynamic powers, both measured at one locked clock. Then the truncated program under cap P reaches the clock the native program reaches under

$$P' = P_0 + (P - P_0)/R, \tag{1}$$

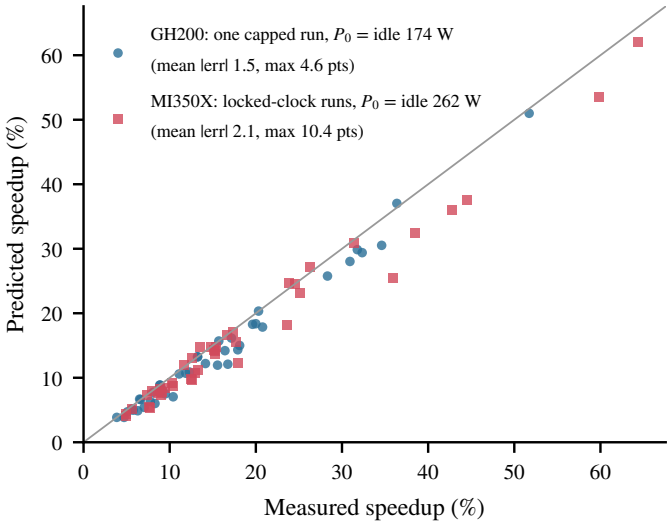


Fig. 1. Predicted against measured GEMM speedups under caps for FP64 (0, 12/27, 35/36 kept fraction bits), BF16 (0, 3) and FP8 (0), with P_0 set to the independently measured idle power (no parameter fitted to these points). GH200: calibrated at the lowest cap (that point excluded); MI350X: from locked-clock power and clock alone.

and its performance is

$$p_r(P) = \sigma\, p_n(P'), \qquad \Delta W = P' - P = (P - P_0)\left(\tfrac{1}{R} - 1\right), \tag{2}$$

capped at the program's clock-limited peak. The equivalent power saving is thus proportional to the dynamic part of the cap, and the whole curve follows from the native curve $p_n(P)$ and two numbers measured once.

The model does not replace measurement of the native program: it needs the native curve $p_n(P)$, the platform's static power P_0 and two numbers for the truncated kernel. What it saves is measuring every truncated variant at every cap. We take P_0 to be the measured idle power, which is independent of the validation data: 174 W for a GH200 module (median while the GPU idles between GEMM runs, three nodes 169–179 W) and 262 W per MI350X GPU (node n02; 265 W on n01). The truncated kernel is characterized in one of two ways. (a) *Locked clock* (MI350X): R and σ from the locked-clock runs of Section III at 1300 MHz, which use no point of the cap sweep. (b) *One cap* (GH200, which we could not clock-lock): $\sigma = 1$ and R from the measured speed at the lowest cap, which is then excluded from the evaluation. Predictions are compared with the measured speedups of the GEMM sweeps at the other caps (GH200: 300–1000 W, 6 caps per format; MI350X: 400–1000 W, 7 caps, node n02).

With P_0 fixed at the idle power, the model predicts the GH200 points with a mean absolute error of 1.5 percentage points (largest 4.6) and the MI350X points with 2.1 (largest 10.4; Fig. 1, Table II). Fitting P_0 on the other formats and predicting the held-out one gives 1.0 and 2.3 points, so the result does not hinge on the choice of P_0; treating all power as dynamic ($P_0 = 0$) roughly triples the error, and on MI350X omitting the clock-stretch term σ raises it from 2.1 to 2.5.

TABLE II

Model error in percentage points of speedup over 42 cap–format points per platform. Idle: P_0 = measured idle power; LOO: P_0 fitted on all other formats and applied to the held-out one; zero: all power treated as dynamic. Last column: largest error for FP64 with 27–36 kept bits.

Platform	P_0	Mean	Max	Max, FP64 27–36
GH200	idle 174 W	1.5	4.6	2.9
GH200	LOO (190–200 W)	1.0	3.6	—
GH200	0	6.1	12.9	—
MI350X	idle 262 W	2.1	10.4	2.9
MI350X	LOO (190–250 W)	2.3	9.4	—
MI350X	0	4.0	17.2	—

The ratios are $R = 0.94$, 0.89 and 0.67 for FP64 with 36, 27 and 0 kept bits, 0.92 and 0.85 for BF16 with 3 and 0 bits and 0.90 for FP8 with 0 bits, with $\sigma = 1.010$–1.053. Errors are smallest where applications operate: for FP64 with 27–36 kept bits the mean error is 0.6–1.7 points on both platforms and the largest 2.9. They are largest where the assumptions break: BF16 with 0 bits at 400 W on MI350X (10.4 points), where the native GEMM barely runs, and FP64 with 0 bits at 500–800 W, which runs at clocks the native GEMM never reaches (2180 versus at most 1890 MHz), so the native curve must be extrapolated. The practical reading is narrower than "predict any cap from one run": once a platform's idle power and an application's native performance–cap curve are known, two locked-clock measurements of a truncated kernel, or one capped run, estimate its whole curve to about two points, and $\Delta W \propto P - P_0$ explains why the equivalent saving grows with the cap even where the relative speedup does not.

V. Method

Mantissa zeroing. For a floating-point operand with p fraction bits (52 for FP64, 7 for BF16, 3 for FP8 E4M3) we keep the top k fraction bits and set the remaining $p-k$ to zero. Sign and exponent are untouched, so the representable range is the format's full range. We compare truncation toward zero (one bit mask), round-to-nearest (add half a unit of the last kept bit, then mask) and stochastic rounding (add a uniform random integer below the weight of the discarded bits, then mask). Ties go away from zero in the shim, the GEMM microbenchmark, Quantum ESPRESSO, DF-MP2 and the Gaussian-process code, and to even in HPL, the LoRA weight loading and LLM inference. We write "truncate to k bits" for keeping k bits either way: unless stated otherwise, the applications (Quantum ESPRESSO, PySCF, DF-MP2, Gaussian process, ExaGeoStat, third-party codes) and HPL round to nearest, while the GEMM microbenchmarks and HPL-MxP truncate toward zero, and the crossover sweeps of both platforms round to nearest on MI350X and truncate on GH200. HPL counts significand bits including the hidden bit, as rocHPL does: its 36 bits are 35 fraction bits. Each element costs one integer AND (plus an add for rounding), which can be fused into the kernel that produces the operand; here HPL runs a separate rounding kernel and the shim makes a copy, and both costs are included in the measured times.

Error model. For operands uniform on $[-1, 1)$ with both A and B zeroed, the relative error of $C = AB$ is

$$\|\hat{C} - C\|_F \,/\, \|C\|_F \approx \alpha\, 2^{-k}, \tag{3}$$

with $\alpha \approx 0.27$ for round-to-nearest, 0.38 for stochastic rounding and 0.70 for truncation ($k \geq 4$; truncation errs about 2.6 times more than round-to-nearest), independent of the inner dimension from 256 to 16384 (measured by splitting the error terms exactly in FP64). Truncation's error is almost entirely a systematic shrinkage: the least-squares slope of $\hat{C}$ against C departs from 1 by about $-0.6 \cdot 2^{-k}$ (-0.59 to -0.64 across k). This matters little for solvers that iterate to self-consistency or correct with a residual, but hurts processes that accumulate bias, which is why the rounding mode matters so much in Sections VII–IX. The relative errors measured on MI350X ($7.8{\times}10^{-12}$ at 35 bits and $2.0{\times}10^{-9}$ at 27 bits, nearest) agree with the model.

On real Llama2-70B layer-40 activations and weights, α = 0.22 (nearest) and 0.66–0.68 (truncation) for k = 2–4, within about 20% of uniform data. On four real FP64 GEMMs of Quantum ESPRESSO Si 1000, α is 0.19–0.32 (nearest) and 0.72–0.81 (truncation). On four real GEMMs of a gold surface, α is nearly constant across k within each GEMM, confirming the 2^{-k} law, but ranges from 0.07 to 9.6 (nearest) across GEMMs with the amount of cancellation: for nearly orthogonal overlaps such as $\langle\psi|\psi\rangle$ the result norm is far below the product of operand norms, amplifying the relative error. The form of the model holds broadly, but the prefactor must be measured per application; self-consistent iteration corrects these errors, which is why Quantum ESPRESSO tolerates 20–28 bits even with $\alpha \approx 10$.

Equivalent power saving. Under cap P, let the zeroed program reach performance $p'(P)$. Its equivalent power saving is

$$\Delta W(P) = P_{\text{native}}(p'(P)) - P, \tag{4}$$

where $P_{\text{native}}(x)$ is the cap at which the native program reaches performance x, linearly interpolated along the measured performance–cap curve. ΔW is the extra power budget per node (GH200) or per GPU (MI350X) the unmodified program needs to match the zeroed one. If p' exceeds every native point we report a lower bound; if the GPU is not power-bound, the saving appears as lower measured power at the same speed. Because performance is concave in the cap, linear interpolation between widely spaced caps (e.g., 800 and 1000 W) slightly overestimates ΔW. Section IV gives a closed form.

Tolerance-driven precision. Many solvers tighten their tolerance as they converge. In plane-wave DFT each Davidson diagonalization receives a threshold *ethr* that tightens from about 10^{-2} to about 10^{-13} during the self-consistent-field (SCF) iteration. We keep

$$k = \text{clamp}(\lceil -\log_2 ethr \rceil + m,\ k_{\min},\ 52) \tag{5}$$

fraction bits (margin m, floor $k_{\min} = 16$), so early iterations use fewer bits and the last ones use what the tolerance requires. This is an empirical rule, not a guarantee: *ethr* is an absolute eigenvalue tolerance (in Ry), not a relative mantissa error, and the margin absorbs the difference. It needs no application model, only a tolerance the application already computes.

VI. Experimental Setup

GH200 system. Fourteen nodes, each with one NVIDIA GH200 480GB superchip (72-core Neoverse-V2 Grace; Hopper GPU with 96 GB HBM3, maximum SM clock 1980 MHz, FP64 tensor-core peak 67 TFLOP/s [56]), connected by 200 Gb/s InfiniBand and managed by Slurm. The module power cap covers GPU, CPU and memory; the driver accepts caps from 200 to 1000 W (default 1000 W), while the datasheet lists 450–1000 W [56], so our caps below 450 W lie outside that range (measured power still follows them). The GPU alone has a separate 900 W limit. The default of 1000 W is also the highest cap, not an uncapped state: native HPL and FP64 GEMM still hit it much of the time. Software: NVHPC 26.5 (CUDA 13.2, cuBLAS 13.4, NCCL 2.29, HPC-X Open MPI 5), driver 615.71.

MI350X system. Two nodes, each with eight AMD Instinct MI350X OAM GPUs (gfx950, 256 compute units, 288 GB HBM3E, maximum graphics clock 2200 MHz) connected by Infinity Fabric, Ubuntu 24.04, ROCm 7.2.3 (rocBLAS, hipBLASLt, RCCL), Open MPI 5.0.7 with UCX, managed by Slurm. The package power limit (PPT0) per GPU is set with `amd-smi` from 1 to 1000 W; the default of 1000 W is the maximum. We use 400–1000 W. Idle power is 244–264 W per GPU, so a 500 W cap leaves about half the dynamic budget.

Workloads. (1) GEMM microbenchmarks with cuBLAS (GH200) and rocBLAS/hipBLASLt (MI350X): FP64 32768×32768×2048; BF16 and FP8 E4M3 32768×32768×k (k = 6144 on GH200, 8192 on MI350X) with FP32 accumulation, on operands uniform on $[-1, 1)$. (2) HPL and HPL-MxP. On GH200: a CUDA port of rocHPL, FP64, 8 nodes, tuned to N = 286720, NB 1024, a 2 × 4 grid, split fraction 0.05 and NCCL fixed at 8 channels with 2 MiB buffers; HPL-MxP (BF16 LU, FP32 accumulation, FP64 iterative refinement, 8 nodes, N = 491520, on a diagonally dominant test matrix). On MI350X, one node of 8 GPUs: rocHPL (N = 540672, NB 1024, 1 × 8, split fraction 0.1, the best native configuration we found earlier) and the same HPL-MxP code (N = 1034752, NB 5504, 4 × 2, 7.6 PFLOP/s native). (3) Quantum ESPRESSO 7.6 GPU [57] (Si 1000-atom supercell at Γ, DGEMM; Au 112-atom surface with k points, ZGEMM), GPU4PySCF 1.8.1 [58], [59] (valinomycin B3LYP/def2-TZVP with density fitting; taxol RHF and DF-MP2) and three non-GEMM FP64 codes (MFEM, SW4, LAMMPS) on GH200; DF-MP2 on MI350X with the three-index tensor of the same taxol calculation built by PySCF 2.14 on the CPU and the contraction on the GPU (GPU4PySCF does not support ROCm; the correlation energy matches PySCF's to $2{\times}10^{-13}$ E_h and GPU4PySCF's on GH200 to $7.5{\times}10^{-8}$ E_h); and a Gaussian-process likelihood on both (Section VIII-E). (4) MLPerf Training Llama 2 70B LoRA with NVIDIA's v5.0 implementation (FP8 weights and

TABLE III
BIT COUNTS, ROUNDING AND SAMPLES PER EXPERIMENT (G: GH200, M: MI350X).

Experiment	Bits k	Removal	Samples
GEMM, fill controls, crossover	fraction bits	G: trunc.; M: trunc. (crossover: nearest)	G: 3 nodes × 2 pairs; M: 8 GPUs × 2 pairs, 2 nodes
HPL	significand bits (36 = 35 fraction)	nearest, ties to even	G: 2 node sets, 6 pairs; M: 2 nodes, 2–4 pairs
HPL-MxP	BF16 fraction bits	truncation	G: 2 node sets, 6 pairs; M: 2 nodes, 4 pairs each
QE, PySCF, DF-MP2, Gaussian process, ExaGeoStat, MFEM, SW4, LAMMPS	fraction bits	nearest, ties away	G: QE warm-up + 6 pairs; DF-MP2, GP 6 rounds; M: DF-MP2, GP 16 per cap
LoRA	FP8 fraction bits	truncation (also nearest, stochastic)	G: 2 seeds, 4 pairs
LLM inference	BF16/FP8 fraction bits	nearest, ties to even	G: 6 models

GEMMs, activations offloaded to Grace memory), 8 GH200 nodes, global batch 8, and inference on six language models.

Implementation. Microbenchmarks and HPL-MxP zero operands in the kernel that produces them; HPL rounds both operands in a separate kernel before each FP64 DGEMM. For unmodified scientific software on GH200 an `LD_PRELOAD` shim intercepts cuBLAS DGEMM/ZGEMM, GemmEx and strided-batched calls and CPU dgemm/zgemm, writes zeroed copies of A and B to scratch buffers (the caller's data are unchanged) and can record exponent histograms and per-row exponent spans of the operands. Quantum ESPRESSO is additionally modified to zero the Davidson vectors in place; DF-MP2 and the Gaussian-process code zero their operands in place; LoRA zeroes the frozen FP8 base weights once after loading. GEMMul8 3.2.0 (MI350X) and cuBLAS emulation (GH200) replace FP64 GEMMs through a hook or the cuBLAS math mode. Table III lists what the bit count means, how bits are removed and how many samples each experiment has.

Measurement protocol. On GH200, nvidia-smi samples power, SM clock, temperature and throttle reasons every 250–500 ms; on MI350X a sampler reads the GPU metrics table of all eight GPUs every 100 ms (4 ms per sweep): package power, the clock of each of the eight XCDs, temperatures, the cap in effect, and two hardware accumulators, a 15.26-μJ energy counter and the residency of power and thermal limiting. Energy on MI350X is the difference of the energy counter across a window, which agrees with the mean sampled power to within 0.1% over long windows. Every measurement starts from a cooled GPU (GH200: below 45 °C; MI350X: junction below 63 °C, with at least 60 s of rest in the paired designs below); within a cap the variants run in ABBA order on the same GPUs. On MI350X all eight GPUs run the same configuration at once, so each window yields eight paired samples. Each sample records the cap actually in effect, and windows whose cap differs from the intended one are discarded (another user's tool reset caps on the GH200 cluster at least twice; the affected measurements were rerun). Hardware power-brake events on GH200 are read from cumulative throttle counters. The first native run of a job is often slower (warm-up), and since ABBA sequences usually start with native, the first pair can overstate the gain (Section VII-D), so the repeat runs start with one uncounted warm-up run. MLPerf comparisons always use the same seed, i.e., the same data order.

A pitfall: chassis fans and multi-variant interleaving. On MI350X at 800 W, a three-variant ABBA sequence of HPL-MxP (native, 4 bits, 3 bits, then reversed) made the 4-bit variant look faster than the 3-bit one, opposite to the second node and to the phase profile. The node's temperature trace shows why: after a hot run the chassis fans speed up, the junction temperature falls by 25 °C within 20 s, the cool-down condition is met at once and the next run starts under high airflow and stays about 12 °C cooler; the run after it, under relaxed fans, is hot again. The middle variant of a three-variant sequence always follows a hot run, so it was systematically cooler and its slowest GPU ran 20–30 MHz faster. No such position effect appears at the other caps (mean temperature differences by position $\leq$ 1.5 °C), and two-variant ABBA blocks are immune because each variant takes each position equally often. We therefore rerun all multi-variant comparisons as pairwise ABBA blocks (reference v v reference) with a minimum rest, and, since even then a run after a hot run can be up to 15 °C cooler, check every MI350X application result with a temperature-adjusted estimate: within a cap, log performance is regressed on the variant and the run's mean junction temperature (Section VII-B). On a power-limited GPU, cooler silicon leaks less and leaves more of the budget for switching, so temperature is a confounder of the same order as the effect we measure (0.1–0.9% per °C).

Power, energy and statistics. On GH200, energy integrates the driver's 1-s averaged module power over the measurement window, summed over nodes. Power in watts in tables and text is the mean of the instantaneous samples in a steady window; over 845 steady windows it agrees with the 1-s average within $\pm$0.6%, but on bursty kernels (e.g., cuBLAS FP64 emulation) single instantaneous readings deviate by -37% to $+41$%, so energy does not use them. While the cap is hit, the 1-s average is often below the cap, so the limiter acts on a shorter, unknown time scale. Module power includes GPU, CPU and memory; under GEMM load the GPU accounts for 88–90% of it at 900–1000 W, falling to 77% at 400 W. On MI350X power is per GPU package and includes its HBM. Each speedup is a paired ratio "zeroed / native – 1" on the same node, cap and ABBA round. Measurements that share a chassis or a job are not independent, so on MI350X we take the independent unit to be one measurement window of all 8 GPUs (GEMM), one run of the whole node (HPL, HPL-MxP) or one round of all 8 GPUs (GP, DF-MP2), report 95% t intervals over these units, and repeat every MI350X experiment on the second node; per-GPU intervals, which are narrower, appear only where stated. On GH200 the units are node–pair samples (GEMM: 3 nodes

TABLE IV

MI350X: SPEEDUP (%) PER NODE, RAW / TEMPERATURE-ADJUSTED, AT THE DEFAULT LIMIT AND AT 500 W. GEMM 95% INTERVALS USE THE 4 MEASUREMENT WINDOWS OF BOTH NODES AS UNITS (8 GPUS AVERAGED PER WINDOW).

	1000 W		500 W	
Result	n01	n02	n01	n02
GEMM FP64, 36 bits	5.2/5.4	5.0/5.5	8.7/8.6	8.6/8.7
both nodes, windows	5.1 ± 0.6		8.6 ± 0.1	
GEMM BF16, 0 bits	16.4/16.6	14.8/15.2	29.8/29.4	31.4/31.6
both nodes, windows	15.6 ± 1.4		30.6 ± 1.5	
GEMM FP8, 0 bits	9.6/9.5	7.9/8.0	17.1/17.2	17.7/17.7
HPL, 36 bits	4.6/4.5	5.1/4.8	7.3/7.2	6.3/6.5
HPL-MxP, 3 bits	4.9/4.8	5.1/5.1	19.5/18.8	19.1/21.3
GP likelihood, 28 bits	7.6	[pending:]	10.8	[pending:]
DF-MP2, 20 bits	10.2	[pending:]	18.8	[pending:]

TABLE V

EVIDENCE CLASS OF EACH APPLICATION RESULT (G: GH200, M: MI350X).

Deployable: in-program, against the unmodified program	Upper bound: interposer, against a copy baseline
GEMM, HPL, HPL-MxP (G, M); DF-MP2 (G, M); Gaussian process (G, M); Quantum ESPRESSO with the tolerance-driven rule (G); MFEM, SW4, LAMMPS (G); LoRA, LLM inference (G)	Quantum ESPRESSO fixed bits (G; −0.4% against the unmodified program); GPU4PySCF DFT (G); ExaGeoStat (G; 0.5–3.1% slower than unmodified)

× 2 pairs; HPL and HPL-MxP: 6 pairs from two node sets; DF-MP2 and GP: 6 rounds from two jobs), which treats pairs within a job or node set as independent and may still be too narrow. Table IV lists the MI350X results per node, raw and temperature-adjusted.

VII. RESULTS: GAINS UNDER POWER CAPS

Two classes of evidence. A gain measured by truncating inside the program and comparing with the unmodified program is what a user obtains; we call it deployable. A gain measured through the GH200 interposer, against a baseline that takes the same copy path without truncating, isolates the effect of truncation but includes none of the copy cost, and relative to the unmodified program it can be a loss. Table V classifies every application result; the conclusions of this paper rest on the deployable ones.

A. GEMM

Zeroing speeds up GEMM at every cap on both platforms, but the gain does not grow monotonically as the cap tightens (Fig. 2). On GH200, FP64 keeping 27 bits is 9.1 ± 1.0, 8.1 ± 0.2, 11.1 ± 0.3 and 8.9 ± 1.0% faster at 500, 600, 700 and 800 W (95% intervals, 6 paired samples per point), an equivalent saving ΔW of 79–162 W; BF16 keeping 0 bits is 11.9–28.3% faster, ΔW from 121 W to above 200 W. On MI350X the default limit already binds, so the gains start there: at 1000 W FP64 keeping 36, 27 and 0 bits is 5.0, 9.1 and 16.7% faster, BF16 keeping 3 and 0 bits 7.4 and 14.8%, and FP8 E4M3 keeping 0 bits 7.9% (node n02; node n01, 16 pairs per point: 5.2, 9.3, 17.3, 7.9, 16.4 and 9.6% with intervals of ±0.2–1.3 points). Toward 400 W the gains grow to 8.9, 15.5 and 65% (FP64), 19.1 and 38.5% (BF16) and 28.2% (FP8), and the equivalent saving per GPU is 42–72 W for FP64 with 36 bits at 600–900 W and 73–147 W for BF16 with 0 bits at 600–800 W. The two MI350X nodes agree within 1.8 points at every common cap and format. The most energy-efficient cap shifts with truncation: on GH200 native FP64 peaks at 82 GFLOP/J at 500 W and FP64 keeping 27 bits at 90 GFLOP/J at 400 W, native BF16 at 950 GFLOP/J at 600 W and BF16 keeping 0 bits at 1159 GFLOP/J at 500 W; on MI350X native FP64 peaks at 63 GFLOP/J at 700 W and FP64 keeping 36 bits at 68 GFLOP/J at 800 W, native BF16 at 988 GFLOP/J at 900–1000 W and BF16 keeping 0 bits at 1169 GFLOP/J at 800 W.

Against quantizing to FP8 (GH200, 575 W, relative to native BF16), BF16 zeroed to 0 bits gives 1.15×, native FP8 E4M3 1.87× and FP8 zeroed to 0 bits 2.12×. On uniform data FP8 E4M3 errs by 3.6%, close to BF16 truncated to 4 bits (4.35%) or rounded to nearest at 3 bits (3.34%). On narrow-range data, including the real Llama 2 70B weights, FP8 wins in both speed and accuracy. On data whose rows span 2^{-24} to 2^{24}, however, FP8 flushes 63% of the values to zero (median elementwise error 100%) while the Frobenius error is only 3.9%; BF16 keeps every value's magnitude even at 0 bits.

TABLE VI

HPL SCALED RESIDUALS (THRESHOLD 16; REPEATED RUNS REPRODUCE EXACTLY). BITS ARE SIGNIFICAND BITS INCLUDING THE HIDDEN BIT (36 BITS = 35 FRACTION BITS), ROUNDED TO NEAREST EVEN. GH200: 8 NODES AT 1000 W, N = 286720 IS THE TUNED CONFIGURATION OF THE CAP SWEEP; MI350X: ONE NODE OF 8 GPUS AT 1000 W. (F): FAILS THE CHECK; BOLD: THE FAILURES THAT RULE OUT 28 BITS.

N	Native	36 bits	32 bits	30 bits	28 bits	24 bits
GH200						
131072	1.5×10^{-4}	0.16	—	—	13.97	456 (F)
196608	1.3×10^{-4}	0.072	—	—	**18.09 (F)**	462 (F)
262144	2.6×10^{-5}	0.042	—	—	14.70	91 (F)
286720	7.6×10^{-5}	0.038	—	—	15.40	—
MI350X						
262144	5.4×10^{-5}	0.46	11.1	84 (F)	**514 (F)**	—
393216	4.6×10^{-5}	0.15	12.0	20.2 (F)	**104 (F)**	—
540672	7.8×10^{-6}	0.27	3.5	17.3 (F)	**102 (F)**	—

B. HPL and HPL-MxP

The HPL residual check determines the usable bit count, it must be checked at several sizes, and the answer depends on the platform (Table VI). On GH200, from 36 to 28 bits the residual grows 86–410×, of the order of $2^8 = 256$, so 28 bits sits less than one bit from the threshold and is not an operating point, whereas 36 bits leaves a margin of about 100× (at least 98×, about seven bits). On MI350X the same 28 bits fails at every size we tried, by 6–32×, and even 30 bits fails; 32 bits passes with a margin of only 1.3–4.6×, and 36 bits passes with 35–100×. The two platforms run different HPL builds and BLAS libraries (blocking and accumulation order of rocBLAS and cuBLAS DGEMM differ), so a bit count validated on one system does not transfer; 36 bits is safe on both.

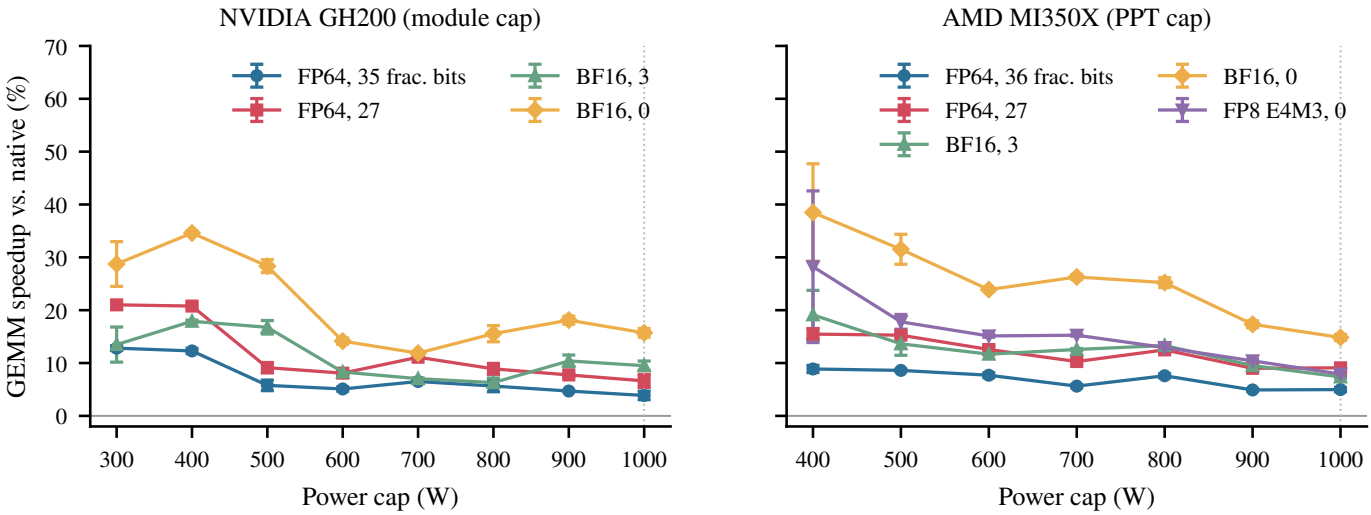


Fig. 2. GEMM speedup from mantissa zeroing under power caps on GH200 (module cap; FP64 32768×32768×2048, BF16 k = 6144; 3 nodes × ABBA) and MI350X (package cap; BF16 and FP8 k = 8192; 8 GPUs × ABBA on node n02). Bars: 95% paired intervals. The dotted line marks each platform's default limit, which is also its highest.

GH200, 8 nodes. We first tuned native HPL at 1000 W. The configuration inherited from an H200 study (N = 262144, NB 1024, 2 × 4, default NCCL) gave 374.8–376.0 TFLOP/s; fixing NCCL at 8 channels with 2 MiB buffers (+4.5%) and raising N to 286720 (+2%) gave 390.2–398.8 TFLOP/s in three runs, and no neighboring setting of N, NB, split fraction or NCCL parameters beat this ≈2% run-to-run spread (24 settings, 26 runs; f = 0.15 and 4 × 2 grids were about 2 and 13% slower). We then swept the caps with this configuration on two node sets: set A (work4–11) with two ABBA pairs per cap, and set B (work3–8, 12 and 13), measured a day later after one uncounted warm-up run, with four pairs per cap. That day the second InfiniBand port of most nodes linked at only 2.5 Gb/s and NCCL and MPI routed part of the traffic over it (157 TFLOP/s on 8 nodes); pinned to the 200 Gb/s port, set B still reached only 365.5 TFLOP/s natively at 1000 W, 7% below set A, although its SM clock was higher (1961 versus 1936 MHz), so more of its time went to communication. Differences between the sets thus mix nodes and network state; every pair is matched within one job.

With 36 bits, GH200 HPL is 3.3–4.7% faster at 500–800 W over the 6 pairs (95% intervals of ±0.1–0.5 points; set A 4.0–5.0%, set B 3.0–4.6%; Table VII, Fig. 3), uses 4.1–4.6% less energy per solve, and is worth ΔW = 42–74 W per node at 500–700 W (at 800 W the interpolation across the 800–1000 W gap overestimates). The speedups stay below the SM clock gains (4.5–7.0%), consistent with communication time that does not shrink with the clock, and set B, which communicates longer, gains less. Pairs within a job agree to within 0.4 (set B) and 0.6 points (set A), but the sets differ by up to 0.9 points at 600–800 W, so two pairs on one node set understate the uncertainty, and the pooled interval, which treats the 6 pairs as independent, may still be too narrow. At 400 W, below the specified range, the gain is 9.9 ± 1.0% (sets A/B: 11.0/9.3%) against only 6.3–6.6% more SM clock, for reasons we have not identified. At the default 1000 W cap both sets use less energy per solve (7.5/8.6%; pooled 8.3 ± 1.1%), but the speedup depends on how far the cap pulls the native clock down: native set A is capped in 75% of the samples at 1936 MHz and also thermally throttled in 15% of them (36 bits: 5%; no other cap or set throttles), and the 36-bit run is 3.9% faster, partly a temperature effect, whereas native set B averages 1961 MHz, 1% below the maximum, and gains only 0.6%. Tuned native HPL reaches 393.9 TFLOP/s at 1000 W on set A (mean of two runs; 398.8 TFLOP/s at best during tuning), about 73% of the 8 GPUs' FP64 tensor-core peak, and the 36-bit run 409.2 TFLOP/s (research results, not official scores). The untuned configuration gave 3.4–8.3% at 400–800 W and 0.7/7.7% at 1000 W. (On set A, 28 bits is 5.7–16.3% faster at 400–800 W, but its residual reaches 15.4 at N = 286720; we list it only as a boundary reference.)

TABLE VII
GH200: DELIVERED PERFORMANCE ON 8 NODES IN TFLOP/S, NODE SET A / B (MEANS PER SET). HPL: TUNED, N = 286720; HPL-MxP: N = 491520, REPORTABLE RATE INCLUDING REFINEMENT. THE 8 GPUS' FP64 TENSOR-CORE PEAK IS ABOUT 536 TFLOP/S (8 × 67 TFLOP/S [56]).

	HPL (FP64)		HPL-MxP (BF16 LU)	
Cap	Native	36 bits	Native	3 bits
400 W	225.0/228.0	249.9/249.2	2076/1922	2340/2128
500 W	288.0/278.2	300.7/291.0	2823/2555	3007/2692
600 W	318.0/304.5	334.0/318.3	3229/2858	3310/2922
700 W	346.2/328.3	362.9/341.3	3449/3039	3519/3094
800 W	370.8/345.8	385.6/356.3	3655/3178	3720/3229
1000 W	393.9/365.5	409.2/367.5	—*	—

Set A: work4–11 (HPL), work4–7 and 10–13 (HPL-MxP), 2 pairs per cap; set B: work3–8, 12, 13, 4 pairs. *Native HPL-MxP trips the chassis power brake at 1000 W on set A; set B was not run there.

MI350X, one node of 8 GPUs. Native HPL (N = 540672)

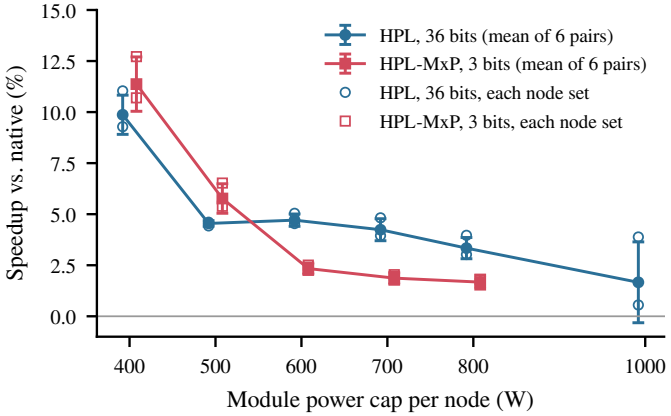


Fig. 3. GH200: 8-node HPL (36 significand bits, tuned, N = 286720) and HPL-MxP (3 kept bits, N = 491520, diagonally dominant test matrix) speedups: mean ± 95% t interval over 6 pairs from two node sets; open markers are each set's mean.

TABLE VIII
MI350X, ONE NODE OF 8 GPUS: NATIVE PERFORMANCE AND SPEEDUP OF THE TRUNCATED VARIANT ON NODES N01 / N02. HPL: N = 540672, TFLOP/s, 36 BITS; HPL-MxP: N = 1034752, PFLOP/s (REPORTABLE RATE), 3 BITS. N01: 2 ABBA PAIRS PER CAP; N02: 1 (HPL) OR 2 (HPL-MxP). ALL RUNS PASS THE RESIDUAL CHECK (HPL 36 BITS: 0.265; HPL-MxP 3 BITS: 0.45–0.54).

	HPL (FP64)		HPL-MxP (BF16 LU)	
Cap	Native	36 bits	Native	3 bits
1000 W	424.9/429.8	+4.6/+5.1%	7.61/7.65	+4.9/+5.1%
800 W	354.8/354.8	+4.9/+4.5%	5.70/5.78	+9.3/+9.2%
650 W	283.3/286.6	+8.0/+6.2%	3.90/3.93	+10.8/+9.7%
500 W	175.9/175.7	+7.3/+6.3%	1.87/1.80	+19.5/+19.1%

reaches 424.9 and 429.8 TFLOP/s on the two nodes at 1000 W, 1.7% apart. Keeping 36 bits, it is 4.6 and 5.1% faster at the default limit, 4.5–4.9% at 800 W, 6.2–8.0% at 650 W and 6.3–7.3% at 500 W (Table VIII), with 4.6–6.8% less energy per solve and ΔW of 39–50 W per GPU at 650–800 W. Over the 8 GPUs of a node, the 36-bit gain at 800 W is thus worth about 350–400 W of node power. Unlike on GH200, the speedups match or exceed the gain in mean clock over the 8 GPUs (4.2–4.8% at 650–1000 W, 6.4–6.5% at 500 W); the synchronous ranks move at the pace of the slowest GPU, whose clock may gain more than the mean, but we have not isolated the cause.

HPL-MxP. HPL-MxP (BF16 LU with FP64 refinement) zeroes the BF16 operands whenever they are written back. Our runs use a test matrix whose diagonal entries equal the sum of the magnitudes of the off-diagonal entries in their row (diagonal dominance), factor the diagonal tiles approximately (`--approx-diag`), and refine with Richardson iteration and an FP32 preconditioner. The HPL-MxP rules allow such diagonally dominant matrices "for testing purposes only" [60], and on such a matrix even coarse L and U factors precondition well, so the stability reported here may be a property of this matrix and cannot be assumed for the generators allowed in official runs. Like HPL, HPL-MxP has a failure boundary, and we use 3 kept bits, stable at all four sizes N = 245760–491520 on GH200 (per-run residuals 1.3–3.8, always 3 refinement iterations) and at N = 176128 and 1034752 on MI350X (0.45–0.54).

On GH200 we measured it on two node sets as well (Table VII): set A with two pairs per cap and set B, the second HPL set with the IB ports pinned, with four pairs after a warm-up run. Its BF16 LU is fast, so communication weighs more than in HPL, and set B's native rate is 7–13% below set A's. At N = 491520, over the 6 pairs, 3 bits is 5.8 ± 0.7% faster at 500 W (sets A/B: 6.5/5.4%) and 1.7–2.3% faster at 600–800 W (intervals of ±0.3 points; the sets differ by only 0.2–0.3 points there), using 3.6–5.9% less energy per solve; at 400 W it is 11.4 ± 1.3% faster. ΔW is 34–45 W at 500–700 W on both sets (at 800 W only set A has a native point above the cap, from another job; we do not report it). Refinement grows from 1 iteration (native) to 3 at every cap, while the speedup of the LU itself shrinks from 13.5–16.0% at 400 W to 5.2–6.0% at 800 W, so above 600 W only about 2% remains after the extra refinement. Keeping 0 bits (set A) gives 7.1–26.0% (ΔW 72–146 W) at N = 491520 with per-run residuals of 10.0–11.1 (a margin of only about 1.5×); at N = 393216 its residual reaches 11.7–13.5, and at N = 245760 refinement jumps from 1 to 52 iterations and performance falls by 70%: passing the residual check does not imply being faster. With fans at full speed, native HPL-MxP at 1000 W trips the chassis hardware power brake (4–10 s each time; in a diagnostic job the brake flag appears in every native run and in no 0-bit run), so we exclude 1000 W from GH200 speedups.

On MI350X, where the same code reaches 7.61 and 7.65 PFLOP/s natively on the two nodes, 3 kept bits raises the reportable rate to 7.98 and 8.04 PFLOP/s at the default limit (+4.9 and +5.1%) and gains 9.2–19.5% at 500–800 W with 5.3–14.6% less energy per solve (Table VIII); the nodes agree within 1.1 points. A phase profile (node n02) shows the trailing BF16 GEMM 6.6–11% faster with 3 bits at 1000–800 W, the residual refinement growing from 2 to 4 steps (1.1 to 2.7 s, about 1.5% of the LU time), and the reportable gain correspondingly 1–2 points below the LU-only gain. Keeping 0 bits does not converge on MI350X (refinement stalls at a residual of 8.8×10^{-10} at N = 176128), so 3 bits is the operating point on both platforms. All of this holds for the diagonally dominant test matrix only: the code's generator always makes the diagonal equal to the row's absolute sum, and we have no implementation of a generator allowed for official runs, so neither the 3-bit operating point nor its gain should be read as a general property of HPL-MxP. [pending: robustness check with exact diagonal-tile factorization (`--exact-diag`) at 1000 and 650 W (job 65)]. One measurement was biased by the fan pitfall of Section VI: on node n01 at 800 W, 4 kept bits appeared faster than 3 (6.34 versus 6.23 PFLOP/s), but all four 4-bit runs ran about 12 °C cooler; on n02, and in the phase profile, 3 bits is faster at every cap (6.29 versus 6.12 PFLOP/s at 800 W). A pairwise rerun on n01 with a minimum rest still showed runs that followed a hot run running cooler and faster (by up to 4.5% for the same variant), so we estimate the effect at equal temperature by regressing log performance on the

variant and the run's mean junction temperature (coefficient −0.4 to −0.5% per °C at 800 W). Adjusted for temperature, 3 bits is +9.0 and +9.1% in the two n01 runs and +9.1% on n02, and 4 bits +4.4 and +6.2%: 3 bits is faster, and the earlier 4-bit figure was a cooling artifact. Every other HPL and HPL-MxP cell changes by at most 0.7 points under this adjustment, because native and truncated runs averaged the same temperature within 1 °C there.

C. Plane-Wave DFT and Quantum Chemistry (GH200)

Plane-wave DFT is tolerant. With 28 kept bits, the total energies of QE Si 1000 and Au 112 move by 1.0×10^{-5} and 2.3×10^{-5} Ry under truncation and by 3.5×10^{-7} and 2.8×10^{-7} Ry under round-to-nearest. The gold surface still converges at 20 bits with nearest rounding (5.5×10^{-5} Ry, both runs); for Si 1000, 20 bits is the failure edge (one run converges with 1.1×10^{-4} Ry, the other aborts at iteration 24), and truncation diverges or slows at 20 bits. Gaussian-basis quantum chemistry needs more bits. GPU4PySCF valinomycin DFT errs by 1.1×10^{-8} E_h at 44 bits; at 36 bits the energy oscillates near 10^{-8} E_h and cannot meet a 10^{-10} convergence threshold, but with the chemically sufficient 10^{-7} threshold 40, 36 and 32 bits converge in the same 12 iterations as native, with a 32-bit nearest error of 1.2×10^{-5} E_h (≈0.007 kcal/mol). At 36–40 bits the error is then set mainly by the threshold (32 bits errs an order of magnitude more), consistent with the ≈37 significant bits found sufficient by Dawson et al. [51].

Under caps, with the shim truncating every GEMM and compared against a baseline that takes the same copy path without truncating, Si 1000 at 550 W with 28-bit nearest rounding is 6.2% faster and uses 6.4% less energy (SM clock 1348→1446 MHz), and Au 112 at 450 W is 4.9% faster with 5.1% less energy (one truncated run each, between baseline runs before and after). These are upper bounds for a library that truncated internally; against the unmodified program the shim is 0.4% slower (Au 112, 450 W) because of its ≈5% copy cost. The in-program version of Section VII-D truncates only Davidson blocks without copies and is the deployable number.

For GPU4PySCF, nsys shows FP64 GEMM kernels at 46% of GPU kernel time and 32% of total runtime for valinomycin DFT (31% and 27% of kernel time for taxol and tamoxifen); the rest is dominated by copies of computed intermediates (36–50%) and integrals. With 32–36 bits needed and FP64 GEMM keeping 35 bits 5.1–6.5% faster at 500–800 W, truncation alone is bounded at about 32% × 5–6.5% ≈ 1.6–2.1% end to end (3.9% at 400 W). This estimate assumes the default grid and GEMM time proportional to flops at 1000 W speed; on a finer grid (level 6), valinomycin measured at 450–550 W is 3.3–3.5% faster (Section VIII-D). Neither number implies that truncation beats INT8 emulation here: the two dominant GEMM shapes favor different methods (Section VIII-B), and choosing by shape bounds the gain at about 3.5% with cuBLAS emulation for the long-k shape, or 6.6% with the less accurate GEMMul8.

TABLE IX
QUANTUM ESPRESSO WITH IN-PROGRAM TRUNCATION (ROUND TO NEAREST) OF DAVIDSON BLOCKS: ONE UNCOUNTED WARM-UP RUN, THEN MEAN ± 95% t INTERVAL OVER 6 INTERLEAVED PAIRS; NATIVE ROWS GIVE THE MEAN OF THE 6 NATIVE RUNS. ENERGY INTEGRATES 1-S AVERAGED MODULE POWER.

System	Setting	Time	Energy	ΔE (Ry)	SCF
Au 112 (450 W)	Native	91.4 s	35.2 kJ	—	28
	Adapt. m=0	−3.6 ± 0.2%	−3.4 ± 0.1%	$< 10^{-8}$	28
	Fixed 24	−2.0 ± 0.2%	−2.1 ± 0.1%	3×10^{-8}	28
Si 1000 (550 W)	Native	205.7 s	108.4 kJ	—	24
	Adapt. m=0	−2.1 ± 0.1%	−2.2 ± 0.1%	$< 10^{-8}$	24
	Fixed 28	−2.0 ± 0.1%	−2.1 ± 0.2%	$< 10^{-8}$	24

D. Tolerance-Driven Bit Counts in Quantum ESPRESSO (GH200)

The adaptive rule truncates the wavefunction blocks before and after applying the Hamiltonian in the Davidson diagonalization. It reproduces the native total energy in every printed digit (10^{-8} Ry) on both systems with the same number of SCF iterations, while cutting time by 2.1–3.6% and module energy by 2.2–3.4% (Table IX); k rises from 16 to 39 on the gold surface and to 42 on Si 1000 as the SCF converges. Each job starts with one uncounted warm-up run: in two earlier jobs on other nodes without it, the first run (a native one) was slower, inflating the gain of the pair containing it. Those jobs gave 4.0% (gold) and 2.4% (Si) after dropping each job's first pair; the 0.3–0.4-point difference from the new runs exceeds either interval and reflects node-to-node variation, so we report the warm-up runs. A margin of $m = 2$, run once in the earlier jobs, gave −3.7 and −1.6%. The energies stay unchanged for three reasons. SCF is a self-correcting fixed-point iteration that already solves early iterations to a loose *ethr*, and the rule keeps the truncation error roughly within what the current Davidson step already tolerates, so later iterations correct it. In the last iterations *ethr* ≈ 10^{-12}–10^{-13} and $k \approx 39$–42, and the total energy is second order in the wavefunction error. And 10^{-8} Ry on a total of about 10^4 Ry is a relative 10^{-12}. Fixed 24 bits is coarser than the threshold late in the SCF and leaves 3×10^{-8} Ry.

On the gold surface the adaptive rule beats fixed 24 bits in all 6 pairs by 1.2–1.8 points with no energy error, but not only through power. It makes Davidson do about 7–9% less work (311 versus 336 calls of the Hamiltonian application), and in one run whose cap had been externally reset to 1000 W, where the code is nearly unconstrained (native 1957–1960 MHz, both truncated variants about 1973 MHz), it was still about 2% faster than native and fixed 24 bits (73.5–73.7 versus 74.8–75.1 s), close to its 1.6-point lead over fixed 24 bits at 450 W. About half of the gold-surface gain thus comes from fewer Davidson iterations, for reasons we have not identified. On Si 1000 the work is unchanged (206 versus 205 calls), so its 2.1% can be read as a power effect, and the rule ties with fixed 28 bits (0.05 ± 0.11 points). Its main value is that no safe bit count must be known in advance: one margin leaves both systems unchanged and matches or beats the best fixed

TABLE X

DF-MP2 (taxol def2-TZVP): speed relative to native (mean ± 95% t interval; GH200: 6 rounds; MI350X: 16 pairs), and the error $|\Delta E|$ of the MP2 correlation energy in E_h (truncated variants round to nearest). MI350X GEMMul8 times exclude a one-time warm-up (Section VIII-B).

Cap	Native	32 bits	20 bits	Emulation
GH200 (emulation: cuBLAS, 32 bits)				
1000 W	39.5 s	+0.5 ± 0.2%	+0.8 ± 0.1%	−35%
700 W	43.9 s	+4.9 ± 0.4%	+7.4 ± 0.4%	−28%
500 W	51.5 s	+4.7 ± 0.2%	+7.7 ± 0.5%	−18%
MI350X (emulation: GEMMul8, 12 moduli)				
1000 W	21.8 s	+5.6 ± 0.5%	+10.2 ± 0.8%	+67.0 ± 0.6%
800 W	26.6 s	+7.4 ± 0.3%	+13.7 ± 0.5%	+61.3 ± 1.2%
650 W	33.9 s	+9.1 ± 0.4%	+16.1 ± 1.0%	+44.0 ± 1.0%
500 W	55.7 s	+9.8 ± 0.3%	+18.8 ± 0.4%	+34.4 ± 0.9%
$\lvert\Delta E\rvert$		—	$\le 5\times10^{-12}$	$\le 2.2\times10^{-8}$	G: 2.2×10^{-8}; M: 7×10^{-12}

TABLE XI

FP64 kernels without matrix units: speedup with 36, 20 and 0 kept bits at the caps given (GH200: 400–700 W; MI350X: 600 W), and power saved at 1000 W with 20 bits (GH200) or 0 bits (MI350X).

Kernel	36 bits	20 bits	0 bits	Saved
GH200				
FMA loop	+2.7–4.8%	+5.6–9.0%	—	138 W
N-body, inputs only	+1.3–2.0%	+2.6–3.9%	—	59 W
N-body, also in loop	—	−7.7 to −8.7%	—	60 W
Memory copy (450 W)	+7.7%	+14.0%	—	70 W
CG, matrix values		≤+1.7%	—	7 W
MI350X (600 W)				
FMA loop	—	+13.6%	+32.9%	118 W
N-body, inputs only	—	+5.2%	—	16 W
Memory copy	—	—	+28.4%	90 W

count, which can only be found after the fact. Unlike Woo et al. and Huang et al., the arithmetic stays on native FP64 units with one-bit granularity, the signal is the Davidson threshold, and the gain comes mainly from clock headroom under a cap (plus, on the gold surface, fewer iterations) rather than a higher-throughput format.

E. A Power-Hungry Application: DF-MP2

The density-fitted MP2 module of GPU4PySCF [58] draws more power than the DFT codes above: on GH200 during MP2 the GPU draws about 800 W and the module 869–876 W, although at 1000 W only 24–53% of the samples hit the cap, and matrix multiplication takes 78% of GPU kernel time, nearly all of it the contraction $g_{ab} = B_i B_j^{\mathsf{T}}$ over slices of the three-index tensor B (FP64 DGEMM whose inner dimension is the auxiliary basis). We zero B in place once after it is built, so every later contraction benefits without copies, and compare with GPU4PySCF's built-in FP32 path and emulation. On GH200 one process runs DF-RHF once and then the variants interleaved (taxol def2-TZVP, 113 atoms, 2228 basis functions, 5477 auxiliary functions; six rounds per cap from two jobs on two nodes; the second ran an uncounted warm-up at each cap first). On MI350X the same tensor (226 occupied and 2002 virtual orbitals, 18.5 GiB), built by PySCF on the CPU, is contracted on each of the 8 GPUs, which run the variants interleaved in one process (two rounds per cap, 16 paired samples).

Keeping 32 bits changes the MP2 correlation energy by 1–5×10^{-12} E_h and 20 bits by 1–2×10^{-8} E_h on both platforms (Table X). On GH200 at 1000 W the truncated variants run at about native speed and draw 61–75 W (32 bits) and 106–127 W (20 bits) less, using 8.4% and 14.3% less energy; under caps the gains are on par with HPL. On MI350X the GPUs draw 978 W and are held at the limit 95% of the time at 1000 W, so the truncated variants are 5.6 and 10.2% faster already there and 9.8 and 18.8% faster at 500 W, with 5.5–16% less energy. The built-in FP32 path on GH200 is faster still (+9.8, +17.9 and +14.6% at 1000, 700 and 500 W; one of its six rounds at 1000 W was only 2.6% faster), and its error of 2.8–5.7×10^{-8} E_h is within chemical accuracy: the tensor's exponent span is narrow (27 binades, i.e., powers of two, overall and a median of 18 within a row), so FP32 fits (Section VIII-D). Emulation behaves differently on the two platforms: cuBLAS emulation on GH200 errs by 2.2×10^{-8} E_h and is 18–35% slower than native at this shape ($m = n = 2002$, $k = 5477$), whereas GEMMul8 with 12 moduli on MI350X errs by 7×10^{-12} E_h and is 34–67% faster than native, faster than any truncation; its lead over 20-bit truncation shrinks from 57 to 16 points between 1000 and 500 W (Section VIII-B).

F. FP64 Beyond GEMM

Ozaki-type emulation applies only to dense matrix multiplication, so we also measured FP64 work that bypasses the matrix units (Table XI). The benefit depends on how many operations separate the truncation point from the multiplier and on what share of the data traffic is truncated. In the FMA loop the inputs feed the multiplier directly; in N-body the position differences regrow full mantissas, and masking the intermediates inside the loop costs more issue slots than it saves; a copy speeds up only when its bandwidth is power-capped; and in CG the vectors dominate the traffic and the indices cannot be truncated. CG keeps the same 490 iterations, with a true residual of 4×10^{-9} at 36 bits.

On MI350X the FMA loop is power-bound even at 1000 W (native held at the limit 65% of the time), so 0 kept bits is 16% and 20 bits 6.2% faster there; the memory copy, bandwidth-bound at 1000 W, runs at the same 5.24 TB/s with 9% less power and becomes power-bound at 600 W, where 0 kept bits is 28% faster. N-body again gains little (4–5% with 20 bits) because its intermediates regrow full mantissas.

Third-party codes (GH200). We zeroed, once at setup, the coefficient arrays that are stored and multiplied in every step in three GPU FP64 codes: MFEM's partial-assembly quadrature data (about 43% of CG memory traffic), SW4's material parameters (about 9%) and LAMMPS EAM tables (cache resident). Without zeroing all three are bitwise identical to the originals or within rerun noise. MFEM (38.27 M unknowns, $p = 3$) is 0.9/2.0% faster with 36/20 bits (nearest) at 450 W and 0.7/1.4% at 550 W, unchanged at 1000 W; the solution changes by at most 2.3×10^{-11}. SW4 (123 M grid points)

is 0.6/1.3% faster at 350 W and 0.3/0.5% at 450 W, with receiver waveforms changing by at most $3.4\times10^{-11}/3.4\times10^{-6}$ (doubling the grid changes them by 4–10%); LAMMPS EAM (16.38 M atoms) moves 0.0–0.4%, within noise. The gain order, MFEM ≥ SW4 > LAMMPS, follows the coefficients' share of memory traffic: kernel-level gains of 4.8–14.0% at 450–500 W are diluted by the rest of the full-mantissa computation. Zeroing fits best when inputs flow directly and in volume into multipliers or data paths.

VIII. When Format Changes Win and When They Fail

A. Range versus Precision

From the exponent histograms recorded by the shim on GH200 (central 99.9%), operands span 12–14 binades in HPL, about 13 in Llama 2 70B weights, 71–93 in Quantum ESPRESSO, 55–64 in GPU4PySCF taxol and 371–690 in valinomycin (down to 2^{-691}, beyond even FP32's range). Every scientific code fills the FP64 mantissa, so how many bits are needed must be found by zeroing experiments, or estimated from the bitwise information content of the data [34]. Placing range against needed bits, plane-wave DFT (a range too wide for FP16 or FP8, modest precision) is where mantissa zeroing has no smaller-format alternative, Gaussian-basis quantum chemistry needs both range and precision, and LLMs and HPL have narrow ranges and can move to smaller formats.

B. Crossover with INT8 Emulation of FP64

INT8 emulation trades FP64 arithmetic for several times as many integer operations on faster units, so its benefit grows with the ratio of INT8 to FP64 matrix throughput. That ratio is about 30 on GH200 (1979 dense INT8 TOPS against 67 FP64 TFLOP/s [56]) and about twice as high on MI350X, and the two platforms indeed sit on different sides of the crossover for large inner dimensions.

GH200. cuBLAS 13 includes an Ozaki-type FP64 fixed-point emulation with a settable number of mantissa bits. On the same node and cap we compare truncated native FP64 with emulation at similar error (32768×32768×2048, 400–1000 W, ABBA). Emulation works in 8-bit slices, $\lceil (b+1)/8 \rceil$ of them for b bits: 24 and 28 bits both use four and give identical results (2.3×10^{-9}), and 32 and 36 bits both use five. On this square-ish shape, three slices (5×10^{-7}) beat truncation to 20 bits by 51–73%; four slices (2×10^{-9}) beat 28 bits by 6% at 1000 W to 23% at 400 W; at 10^{-11} truncation (about 36 bits) is 11–28% faster than five slices; and at 10^{-13} and below truncation is more than 40% faster. The crossover is thus near 28–32 needed bits. These comparisons truncate toward zero; rounding to nearest zeroes the same bits for one extra addition and errs about 2.6 times less (about 1.4 bits), so at equal accuracy rounding needs about one bit fewer and the crossover moves slightly toward truncation. GEMMul8 on GH200 is version 3.5.1.

The crossover depends on shape. Intercepting all 45,574 GEMMs of the valinomycin DFT run on GH200 reveals two shapes that each carry 35% of the flops (Table XII). With $n = 300$, emulation cannot amortize its splitting and scaling: even at 28 bits it runs at 0.47–0.57× native speed, and truncation wins at every accuracy. With $k = 76800$ the fixed costs amortize: emulation is still 21–22% faster than truncation at the 10^{-11} level, and GEMMul8 is nearly twice as fast at 10^{-9}. Within one application the best choice is per call: truncation for tall-skinny shapes, emulation for long k.

TABLE XII
GH200: the two GEMM shapes that each carry 35% of valinomycin DFT flops: tall-skinny 757248×300×2958 and long-k 2958×2958×76800 (TFLOP/s; relative error in parentheses; GEMMul8 errs by 1.0×10^{-10} and 8.2×10^{-10} on the two shapes, 10–80 times more than truncation to 36 bits).

	Tall-skinny		Long k	
Method	1000 W	575 W	1000 W	575 W
FP64 native	51.6	36.8	50.6	36.2
Truncation, 36 bits (1.0×10^{-11})	54.0	38.9	53.7	38.0
cuBLAS emu., 32 bits (9.7×10^{-12})	19.5	16.6	65.4	46.0
cuBLAS emu., 28 bits (2.3×10^{-9})	24.2	20.9	89.9	65.6
GEMMul8, 10 moduli	17.9	16.7	102	75.6

A shape sweep and a predictive model (GH200). We fixed $m = 8192$ and swept $n \in \{256, 1024, 4096, 8192\}$ and $k \in \{256, 1024, 4096, 16384, 81920\}$ at 1000, 575 and 450 W, two rounds each (480 records; Fig. 4). Truncation to 36 bits beat native in all 60 shape–cap combinations (3.8–11.9% at 450 W, 3.6–5.9% at 575 W, 0.5–4.8% at 1000 W). cuBLAS emulation with 32 bits wins roughly when $n \geq 4096$ and $k \geq 16384$ (and at $n = 1024$, $k = 81920$ at 1000 and 575 W) (at best 0.74× truncation time at 1000 W), and its winning region shrinks at lower caps: 5 of 20 shapes at 1000 and 575 W, 2 at 450 W. Where the emulation itself is not power-bound, however, lower caps favor it: at $n = 8192$, $k = 4096$ its time relative to truncation falls from 1.17 to 1.08 to 1.05 as the cap goes from 1000 to 575 to 450 W. The four values of n were measured on different nodes: each ratio is paired on one node, but shape effects and node differences cannot be separated. GEMMul8 with 10 moduli needs only 0.47–0.80× the truncation time when $n, k \geq 4096$, at an order of magnitude larger error ($\approx 10^{-10}$ versus 10^{-11}), and is up to 4.5× slower when n or k is small. Emulation's extra work is input splitting ($\propto mk + nk$) and output accumulation ($\propto mn$); dividing by the mnk multiply gives

$$T_{\text{emu}}/T_{\text{trunc}} = A + B\,(1/m + 1/n) + C/k, \tag{6}$$

where A is the ratio of per-flop speeds. Least-squares fits per cap (about 20 points for three parameters, so R^2 is only descriptive) reach $R^2 = 0.98$–0.99 for GEMMul8, versus 0.86–0.89 for a single effective size $1/(1/m + 1/n + 1/k)$; cuBLAS emulation reaches only 0.88–0.89. On the two held-out valinomycin shapes the model picks the winner in 8 of 8 cases, all far from the crossover, so this is a loose test; the model is an empirical description of these measurements.

MI350X: the cap moves the crossover. On MI350X we compared truncation (35 and 27 bits, round to nearest) with GEMMul8 3.2.0 (8–14 moduli) for $m = n = 32768$ and $k = 1024$–8192 at 1000, 700 and 500 W (Fig. 5). Truncation to

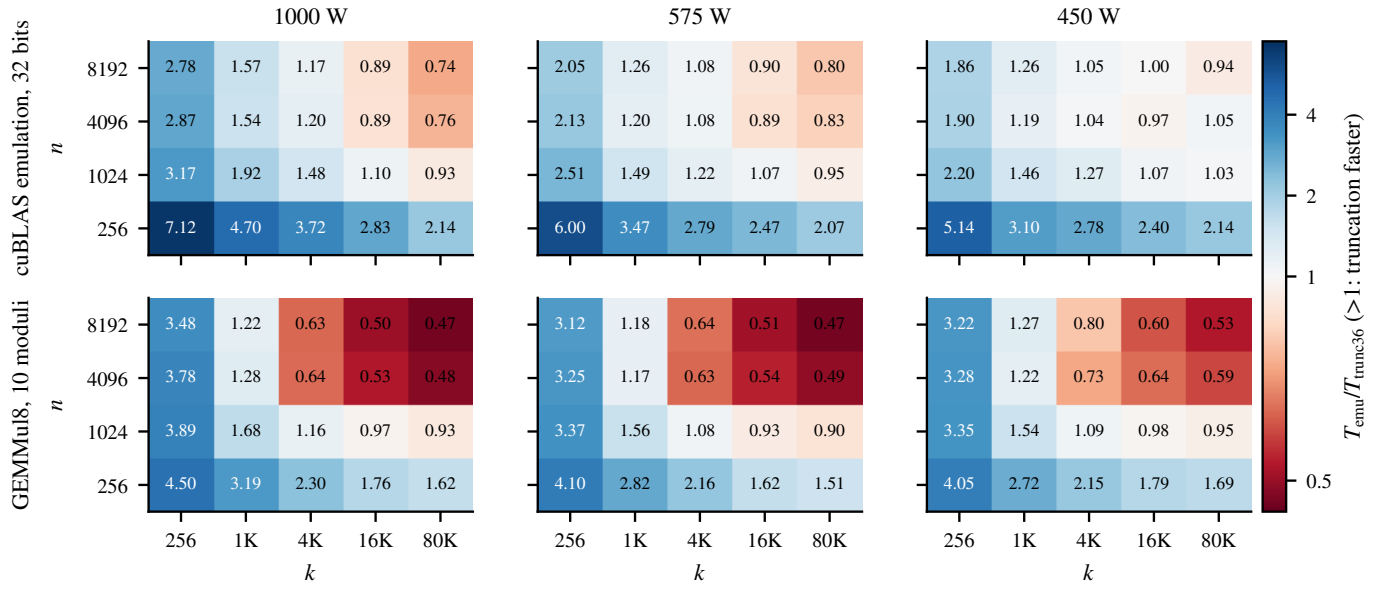


Fig. 4. GH200: time of INT8 emulation relative to truncation to 36 bits, $T_{\text{emu}}/T_{\text{trunc36}}$ (m = 8192; blue: truncation faster). Top: cuBLAS emulation with 32 bits (error comparable to truncation, $\approx 10^{-11}$); bottom: GEMMul8 with 10 moduli (about ten times larger error, $\approx 10^{-10}$); columns: module caps. Means of two rounds.

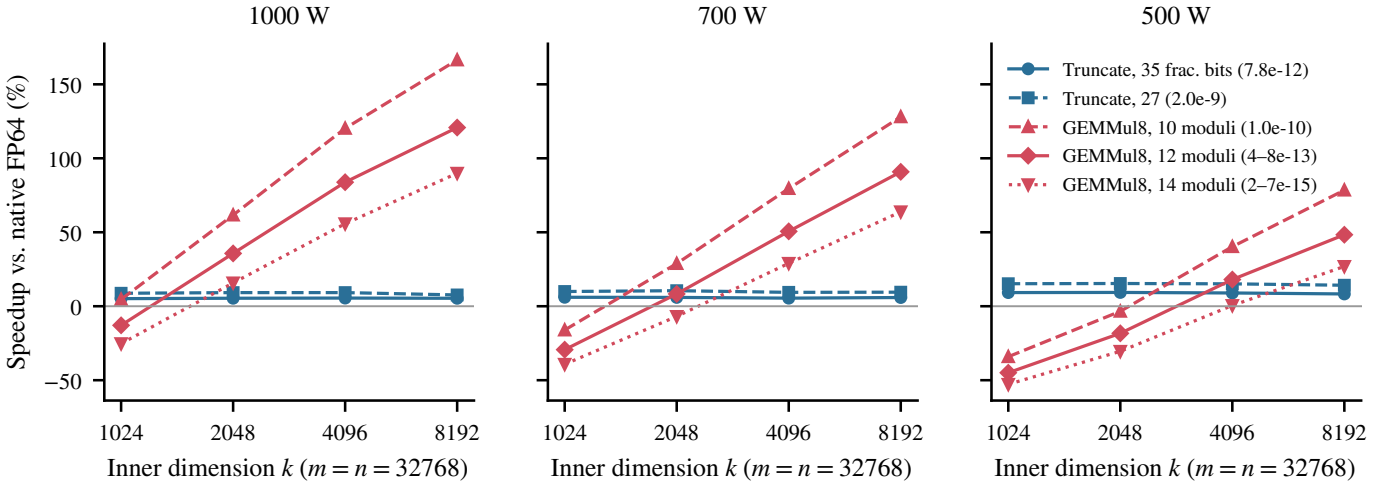


Fig. 5. MI350X: speedup over native FP64 of truncation (round to nearest) and GEMMul8 for $m = n = 32768$ as a function of the inner dimension k, at three package caps (8 GPUs × ABBA, node n02). Relative errors in the legend. A tighter cap lowers every emulation curve and raises every truncation curve.

35 bits (7.8×10^{-12}) is 5.1–5.5% faster than native at 1000 W and 8.3–9.3% at 500 W for every k. GEMMul8 with 12 moduli (4×10^{-13}–8×10^{-13}, more accurate than 35 bits) is 13% slower than native at $k = 1024$ but 36, 84 and 121% faster at $k = 2048$, 4096 and 8192 at 1000 W. At 500 W those gains fall to −45, −18, +18 and +48%. The crossover thus moves with the cap: at $k = 2048$ emulation wins at 1000 and 700 W and truncation at 500 W, and at $k = 1024$ truncation wins everywhere. The direction is the same as GEMMul8's on GH200 (Table XIII) and follows from power: GEMMul8 draws the full limit, and the extra integer work it does per useful FP64 operation is paid for with clock when power is the constraint, whereas truncation removes work from the power budget. The ratio of INT8 to FP64 throughput sets where the crossover lies; the cap sets which way it moves.

The power of the emulation itself changes the winner. On GH200, GEMMul8 draws about 970 W at a 1000 W cap and at 450 W its clock drops to 650–720 MHz, where truncation becomes faster; cuBLAS fixed-point emulation with 28 bits draws only about 706 W at 1000 W and gains relative ground at lower caps on the square shape (Table XIII). Which way a tighter cap tips the balance thus depends on how much power the emulation itself draws.

Applications: memory and warm-up. In DF-MP2 on MI350X ($k = 5477$), GEMMul8 with 12 moduli is 34–67% faster than native and beats truncation at every cap (Table X),

TABLE XIII
GH200: SQUARE GEMM (32768×32768×2048) IN TFLOP/S BY CAP ON ONE NODE, NATIVE FP64 FIRST AND THE OTHER METHODS ORDERED BY RELATIVE ERROR.

Method (error)	1000 W	575 W	450 W
FP64 native	59.2	43.2	35.8
Trunc., 36 bits (1.0×10^{-11})	61.8	45.5	38.6
GEMMul8, 10 moduli (1.0×10^{-10})	70.8	51.4	34.7
GEMMul8, 9 moduli (1.6×10^{-9})	78.2	56.6	38.2
cuBLAS emu., 28 bits (2.3×10^{-9})	67.8	55.2	47.5
Trunc., 28 bits (2.6×10^{-9})	63.2	46.6	39.6
FP32 SGEMM (8.1×10^{-7})	51.2	40.4	34.6
3xTF32 (4.8×10^{-6})	96.6	72.0	56.4

its lead again shrinking as the cap tightens. Two costs appear only in applications. First, each new GEMM shape costs GEMMul8 a large one-time setup: DF-MP2 forms 226 shapes of growing n, and the first pass through them takes 142 s instead of 13 s (the native contraction takes 22 s; pre-allocating the workspace does not help). Programs that call many distinct shapes once lose; the times in Table X exclude this warm-up. Second, emulation needs workspace. In HPL on MI350X the matrix of our best native configuration ($N = 540672$) occupies about 292 GB per GPU, and GEMMul8's 15 GB workspace no longer fits, so every emulated run aborts; with 12 moduli even $N = 491520$ does not fit. HPL's score grows with N, so emulation must give up problem size before it can start, and the grid matters too: on the 1×8 grid, where each GPU holds all N rows, GEMMul8 requests 100–114 GB of workspace already at $N = 458752$ and again aborts. On the configuration used for GEMMul8's best MI350X result ($N = 491520$, NB 4096, 4×2), 10 moduli pass the residual check (1.79, against 0.03 for 36-bit truncation) and run at 576.8 TFLOP/s at 1000 W, 48.5% above native in this configuration and 36% above the best native configuration (424.9 TFLOP/s), reproducing an earlier record of 579.7 TFLOP/s on this hardware. The cap again erodes the lead: over native in the same configuration GEMMul8 gains 48.5, 41.7, 32.8 and 13.9% at 1000, 800, 650 and 500 W, truncation 3.8, 3.9, 6.6 and 6.9%; against the best truncated run (444.6 and 188.8 TFLOP/s at 1000 and 500 W) GEMMul8 leads by 30% at 1000 W and by only 4% at 500 W (two pairs per cap; temperature-adjusted values within 3 points). With 12 moduli it runs out of memory at this N too. [pending: native vs 36 bits vs GEMMul8 12 moduli at $N = 458752$, NB 4096, 4×2 (job 64)].

C. FP32 and 3xTF32: Accumulation, Not Range (GH200)

3xTF32 (our own implementation, in the shim and in a benchmark script, not the CUTLASS one) is the fastest method in Table XIII but has only FP32-level accuracy, and its error grows with the inner dimension: on 4096×4096 it is 6.4×10^{-7}, 4.8×10^{-6} and 3.8×10^{-5} at $k = 256$, 2048 and 16384, whereas FP32 SGEMM grows as $\sqrt{k}$ (2.9×10^{-7}, 8.1×10^{-7}, 2.3×10^{-6}) and FP64 truncation is independent of k. The measured log slope of about one (7.5–7.9× per 8× in k) points to a systematic error in TF32 tensor-core accumulation rather than random rounding, consistent with the round-toward-zero accumulation Fasi et al. found on V100, T4 and A100 [61] (bit-accurate Hopper models: [62]). At $k = 2048$ its error is 18 times that of FP64 rounded to nearest at 20 bits (2.6×10^{-7}) and 7 times that of FP64 truncated to 20 bits (6.7×10^{-7}).

Quantum ESPRESSO's operands fit in FP32's range, so we replaced every FP64 and complex FP64 GEMM with single precision through the shim (inputs to FP32; 3xTF32 additionally splits into big and small parts multiplied three times on TF32 tensor cores; C combined in FP64). On the gold surface at 450 W (2 interleaved runs each), FP32 aborts at SCF iteration 26 and 3xTF32 and TF32 at iteration 2 when the Davidson bands fail to converge, and the integrated charge drifts from 1232 electrons (FP32 +0.012, 3xTF32 +0.074); FP64 rounded to 24 and 20 bits differs from native by 8.1×10^{-6} and 5.5×10^{-5} Ry and converges in 28 iterations. Si 1000 at 550 W behaves the same: FP32 aborts at iteration 14 and 3xTF32 at 9, with 4000.18 and 4001.52 electrons instead of 4000. The difference is accumulation: truncated FP64 still accumulates in FP64, while Quantum ESPRESSO's inner dimension is the number of plane waves (95386 on the gold surface plus a real GEMM with $k = 4.3\times10^{6}$; 375286 for Si 1000), where 3xTF32 already errs by 5.6×10^{-5} to 2.2×10^{-4}, the same order as plain TF32. FP32 GEMM on Hopper is also slower than FP64 tensor cores (on the gold surface about 1.5× per SCF iteration, estimated). Replacing every GEMM with single precision thus fails for this long-inner-product solver even though the range fits; mixed precision that uses FP32 only in selected steps, as Woo et al. do [49], is a different matter. FP64 truncation is the reduced-precision choice here that leaves the algorithm unchanged.

D. Exponent Span Limits Format Changes

Truncation keeps every element's 11-bit exponent and changes only its mantissa; format changes must handle the exponent. Per-tensor scaled formats (FP32, FP8) need the whole tensor inside the format's range, and Ozaki-type INT8 emulation slices each row (column) relative to its maximum, so with k kept bits elements more than 2^{-k} below the row maximum are lost. The exponent span of the data therefore limits format changes, whereas truncation is unaffected.

Synthetic data. On GH200, on 8192×8192×16384, a shape where emulation is faster at narrow spans, we draw element exponents uniformly over S binades (1000/575 W, two rounds; Fig. 6). Truncation to 36 bits keeps a median elementwise error of 1.0–1.1×10^{-11} and a speedup of 4.4–6.3%. cuBLAS emulation with a fixed 32 bits degrades by 4–9× in median and 5–19× at p99 ($S = 16$–256), the equivalent of 2–3 fewer bits, which costs a whole extra 8-bit slice to recover. Its dynamic mode, which targets FP64-equivalent accuracy, raises the bit count from 58 to 301 and falls back to native FP64 beyond its maximum, but with the extra work of determining the span it takes 1.6–2.6× the truncation time (2.6× at $S = 16$); FP32 overflows at $S = 256$. GEMMul8 is barely affected on uniformly spread data. If the spread is only between rows (one scale per row), emulation is unaffected while FP32 still over-

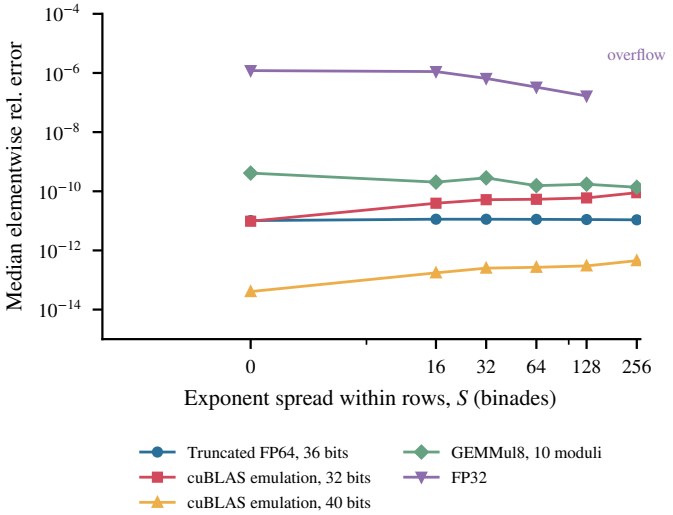


Fig. 6. GH200, synthetic exponent spread: elements of 8192×8192×16384 operands have exponents uniform over S binades within each row (1000 W).

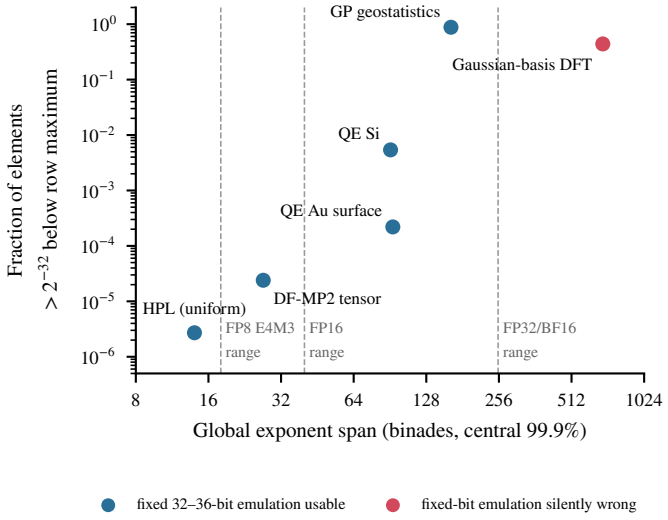


Fig. 7. GH200: global exponent span (central 99.9%) against the flop-weighted fraction of elements more than 2^{-32} below their row maximum, with the measured outcome of fixed 32–36-bit cuBLAS emulation (e.g., HPL: residual 0.197, passes; Gaussian-basis DFT: 52.5 mE_h error; Gaussian process: global span 162, estimate unchanged). All points are measured; coordinates and their sources are listed in `gh200/results/range_map_points.csv`.

TABLE XIV
GH200, REAL DATA (1000 W): FLOP-WEIGHTED FRACTION OF NONZEROS MORE THAN 2^{-32} BELOW THEIR ROW MAXIMUM, AND THE TOTAL-ENERGY ERROR OF EACH METHOD. THE "NEAREST" ROWS KEEP 36 OR 32 BITS WITH ROUND-TO-NEAREST; CUBLAS EMULATION WITH 36 AND 32 BITS USES FIVE SLICES, WITH 40 BITS SIX; AUTO: ITS AUTOMATIC MODE (AT MOST 79 BITS). THE AU 112 FRACTION COMES FROM A TWO-ITERATION BENCHMARK INPUT, ITS ERRORS FROM THE FULLY CONVERGED INPUT.

	Valinomycin	QE Si 1000	QE Au 112
Below 2^{-32}	24–44%	0.24–0.54%	0.008–0.02%
Nearest, 36 bits	9.6×10^{-7} E_h	0 Ry	0 Ry
Nearest, 32 bits	1.2×10^{-5} E_h	0 Ry	4×10^{-8} Ry
cuBLAS emu., 36/32	**5.25×10^{-2} E_h**	6×10^{-8} Ry	5×10^{-8} Ry
cuBLAS emu., 40	8.4×10^{-8} E_h	—	—
cuBLAS emu., auto	4×10^{-9} E_h	—	—
GEMMul8, 10 moduli	diverges	out of memory	3.7×10^{-7} Ry

flows. On MI350X the same kind of sweep (8192×8192×4096, S = 0–64 per element, 32 and 64 per row) leaves GEMMul8 with 10, 12 and 14 moduli and truncation to 35 and 27 bits unchanged in median and p99 elementwise error and in speed (12 moduli: median 5.7×10^{-13}–8.0×10^{-13}, p99 at most 4.4×10^{-11}). Uniform synthetic spreads thus do not trigger the failure on either platform; it needs the structure of real data described next.

Real data. The shim samples 256 rows of op(A) and 256 columns of op(B) per GEMM and records the fraction of nonzeros more than 2^{-32} below their row maximum, weighted by flops (validated against theory on synthetic data). In Gaussian-basis DFT the atomic-orbital values on the grid decay exponentially with distance, and in the worst shape 87% of elements are more than 2^{-32} below their row maximum. There, fixed 36-bit cuBLAS emulation (five slices, like 32 bits) lets the SCF converge normally in 12 iterations to a total energy that is off by 52.5 mE_h (≈33 kcal/mol), and GEMMul8 diverges; rounding to the same 36 bits errs by 9.6×10^{-7} E_h (Table XIV). What fails is the user-fixed bit count. With 40 bits (six slices) the emulation is correct (8×10^{-8} E_h) but takes about 1.5× the time of truncation to 36 bits, and cuBLAS's automatic mode, capped at 79 bits, is correct too (4×10^{-9} E_h): 89% of the flops would need 80 or more bits and fall back to native FP64, the rest is emulated with 72–79 bits, and the run takes 2.1× the native time. On wide-span data emulation is therefore not impossible, but not worthwhile. HPL sits at the other extreme: only 2.7×10^{-6} of its flop-weighted elements are more than 2^{-32} below their row maximum, and fixed 36-bit emulation passes the residual check (0.197, against 0.057 for HPL's own rounding to 36 bits) but runs at 0.65× native speed on one GPU (N = 81920; the automatic mode keeps the native residual level, 1.9×10^{-6} against 1.7×10^{-6}, at 0.35×). On Quantum ESPRESSO, with narrow per-row spans, fixed 36-bit emulation errs by only 5–6$\times10^{-8}$ Ry. Uniform synthetic spreads understate the problem, since in real data many elements of a row lie far below its maximum. A wide per-row span, which the shim measures cheaply beforehand (Fig. 7), is thus a risk indicator: a necessary but not sufficient condition for failure, and tile-level mixed precision can also handle such data (Section VIII-E). DF-MP2 (Section VII-E) sits at the narrow end, where FP32 works and GEMMul8 on MI350X is accurate.

Wide range, partly power-bound DFT. These DFT runs draw only 650–690 W at 1000 W, but are power-bound below that. For valinomycin (grid level 6) at 550 and 450 W, rounding to 36 bits through the shim is 3.3–3.5% faster than the copy baseline with 2–3% less energy and an error of 9.5×10^{-7} E_h; emulation must keep 40 bits to stay correct, which is then about ten times more accurate (8.4×10^{-8} E_h) but takes 1.48–1.53× the baseline time (two rounds each).

TABLE XV

GAUSSIAN-PROCESS LIKELIHOOD (n = 80000, SIX RANGE PARAMETERS): LARGEST LOG-LIKELIHOOD DEVIATION FROM NATIVE AND SPEED RELATIVE TO NATIVE (MEAN ± 95% t INTERVAL; GH200: 6 ROUNDS; MI350X: 16 PAIRS; TRUNCATION ROUNDS TO NEAREST). THE LOG-LIKELIHOODS ARE 1.2–1.7×10^5. MI350X INTERVALS ARE PER GPU; WITH ROUNDS AS UNITS SEE TABLE IV. [a] AT n = 80000 GEMMUL8 RETURNS SUCCESS BUT AN INVALID TRAILING UPDATE; SEE TEXT.

	Trunc. 36	Trunc. 28	Emulation	FP32
GH200 (emulation: cuBLAS, 36 bits)				
max $\lvert\Delta\ell\rvert$	2.4×10^{-5}	7.2×10^{-4}	2.2×10^{-3}	87
1000 W	+3.0 ± 0.3%	+3.8 ± 0.4%	−41%	−8.2%
700 W	+5.0 ± 0.1%	+7.9 ± 0.2%	−31%	+3.2%
500 W	+4.8 ± 0.1%	+7.5 ± 0.1%	−21%	+3.9%
MI350X (emulation: GEMMul8)				
max $\lvert\Delta\ell\rvert$	1.9×10^{-5}	5.3×10^{-3}	wrong[a]	—
1000 W	+4.0 ± 0.2%	+7.6 ± 0.2%	—	—
800 W	+3.9 ± 0.4%	+6.1 ± 0.7%	—	—
650 W	+5.4 ± 0.7%	+7.9 ± 1.2%	—	—
500 W	+6.7 ± 0.4%	+10.9 ± 0.6%	—	—

E. Wide Range and Power-Bound: Gaussian-Process Geostatistics

Maximum-likelihood estimation of a Gaussian process in geostatistics factors an $n\times n$ Matérn covariance matrix by Cholesky at every likelihood evaluation. As in HPL, almost all work is FP64 matrix multiplication, but the covariance decays exponentially with distance. We implemented an ExaGeoStat-like kernel, in CUDA with cuBLAS on GH200 and in HIP with rocBLAS/rocSOLVER on MI350X: n = 80000 random 2-D locations, Matérn-3/2 covariance (nugget 10^{-4}), blocked Cholesky (n_b = 1024, trailing updates by one FP64 DGEMM per panel), and the log-likelihood at six range parameters (0.02–0.08). Each panel L_{21} keeps k bits before the trailing update, in one fused pass that walks the panel in memory order (an earlier version with two separate, uncoalesced passes left 28 bits at 5.2–5.7% under caps on GH200, so the reduction itself must be cheap). On GH200 we compare with cuBLAS emulation and FP32 (1000/700/500 W, six interleaved rounds from two jobs on the same node); on MI350X each of the 8 GPUs runs the variants interleaved (16 pairs per cap). Native draws 953–970 W of module power at 1000 W on GH200 and is capped 91–94% of the time; on MI350X it draws 924 W per GPU and is held at the limit 86% of the time. The sampled panel rows span 162 binades (central 99.9%), and 88% of the panels' nonzeros are more than 2^{-32} below their row maximum.

A deviation of 7.2×10^{-4} (GH200) or 5.3×10^{-3} (MI350X, different random locations) at 28 bits, a relative 3×10^{-8}–4.5×10^{-8}, is far below what matters for estimation (Table XV). On GH200 at 1000 W, where this native factorization is capped 91–94% of the time, 28 bits is 3.8% faster (36 bits 3.0%), but the 1000 W gain depends on the setting: with the Morton ordering used below (n = 60000), where native is capped in only 19–54% of the samples, 28 bits gains +0.0/+0.1%, and the maximum-likelihood fit below gains 1.8–2.3%. On MI350X the default limit binds, and 28 bits is 7.6% faster at 1000 W and 10.9% at 500 W, with 5.7–9.8% less energy. FP32's deviation makes it unusable for estimation; it may stem from the conditioning of the covariance, which worsens with n (at n = 8000 FP32 still finds the same estimate). cuBLAS emulation on GH200 is less accurate than 28-bit truncation and slower, partly because the trailing update's inner dimension equals the block size, 1024, a shape that already favors truncation in the crossover model (Section VIII-B) and in the MI350X sweep, where GEMMul8 loses to truncation at k = 1024 under every cap. The per-row span is wider than in the DFT case, yet emulation does not go wrong, only loses accuracy: a wide per-row span is necessary but not sufficient for emulation failure, which also depends on how much the dropped small elements contribute.

On MI350X, GEMMul8 failed in a different way. At n = 80000 every one of its 256 runs stopped with a matrix that was no longer positive definite: the hooked DGEMM of the first trailing update ($m = n$ = 78976, k = 1024) returned success but an invalid result, with no diagnostic, with or without pre-allocated workspace. With 12 or more moduli the same code gives the native log-likelihood to 10^{-9} relative at n = 2048–65000, so the failure depends on size, between $m = n$ = 63976 and 78976; we have not located its cause in the library and report it as an observation about the GEMMul8 3.2.0 HIP build we used, not about emulation in general. It nonetheless illustrates the operational risk of replacing a BLAS call wholesale: a numerically safe configuration can fail silently on a larger instance. At n = 65000, where it is correct, GEMMul8 with 14 moduli takes 4.8 s per likelihood against 3.7 s natively at 1000 W, i.e., it is slower, as expected for k = 1024 (Section VIII-B). [pending: full comparison at n = 64000: native, 36 and 28 bits, GEMMul8 12 and 14 moduli, four caps (job 63)].

A one-parameter maximum-likelihood fit (GH200). To check that the estimate itself is unchanged, we fitted the range parameter with variance and smoothness fixed: golden-section search over its logarithm, 17 likelihood evaluations per variant (n = 60000; 1000 and 700 W; two interleaved rounds). Truncation to 36 and 28 bits and cuBLAS emulation return the native estimate $\hat\theta$ = 0.05007952 in every printed digit, with maximized log-likelihoods within 4.8×10^{-6} (36 bits) and 2.3×10^{-4} (28 bits) of native; FP32 moves $\hat\theta$ by 0.49% and loses 0.72 in log-likelihood. At 700 W the fit is 7.1–7.3% faster with 28 bits and 4.8–4.9% faster with 36 bits, using 6.6–6.8% and 4.5–4.8% less energy; at 1000 W it is 1.8–2.3% and 1.6–2.2% faster with 9.1–9.9% and 3.5–5.6% less energy; cuBLAS emulation is 32–41% slower. Of the three whole-matrix choices we tested (FP32, 36-bit cuBLAS emulation and truncation), only truncation is both fast and accurate on this wide-range, power-bound workload.

Composing with tile mixed precision (GH200). The mixed-precision Cholesky factorizations of the ExaGeoStat line change the format of whole tiles instead (Section II). To see whether truncation still helps there, we ordered the locations along a Morton curve and added such a variant to our kernel: in the trailing update, tiles below each column's FP64 extent, chosen by the tile-norm rule with a required accuracy

of 10^{-9} as in [47], use FP16 inputs with FP32 accumulation on the tensor cores (52% of the tiles at range 0.02, 3% at 0.08). Over the six ranges (n = 60000, two ABBA rounds) it runs 1.60–1.63× faster than native FP64 at 1000, 700 and 500 W, with log-likelihood deviations up to 0.043. Truncating its remaining FP64 tiles to 28 bits makes it another 1.9% faster at 1000 W and 7.0% and 6.1% faster at 700 and 500 W, with an unchanged deviation, so truncation composes with format changes rather than competing with them. Zeroing the FP16 operands to 5 mantissa bits as well added nothing.

ExaGeoStat itself (GH200). We also built ExaGeoStatCPP (StarPU and Chameleon tile Cholesky with cuBLAS 13.6; tile size 1920) and ran its MLE (N = 60000, six BOBYQA likelihood evaluations per run, data generated once natively and read by every run) with the shim on its FP64 tile GEMMs. Matérn covariance generation runs on the CPU and takes about 9 s per evaluation against 2.3 s of GPU factorization at 1000 W, and the factorization is partly power-limited even there (GPU power about 620–670 W on average, frequent power-cap events). Relative to a baseline that copies the operands without truncating, the factorization is 3.0% (36 bits) and 4.0% (28 bits) faster at 1000 W, 5.0% and 8.0% at 600 W, and 5.6% and 9.4% at 450 W (means of two ABBA rounds; at 450 W the rounds differ widely, +6.9 and +11.9% with 28 bits at 707 and 926 MHz, because the CPU shares the module cap while generating the covariance); with 28 bits the mean SM clock during factorization rises from 1913 to 1975 MHz at 1000 W and from 1166 to 1305 MHz at 600 W. The log-likelihoods change by at most 1.2×10^{-10} and 9.8×10^{-8} relative. The shim's own copy costs 7–9%, which leaves it 0.5–3.1% slower than unmodified ExaGeoStat, so the gain needs an implementation inside the library. ExaGeoStatCPP's mixed-precision option (`--computation=mixed`) is only an enumeration value without an implementation, so we could not compare with its own mixed precision.

F. Truncation Inside Emulation

Truncation and emulation are not exclusive. GEMMul8 converts each FP64 row into integers; if the operands carry only k significant bits, the low-order integer slices are largely zero and the INT8 matrix engines toggle less, so under a cap the emulation itself might run faster. On MI350X ($32768^2 \times 4096$, 8 GPUs × ABBA) the effect exists but is small: with 12 moduli, truncating the inputs to 19 bits raises the speedup over native from 18.4 to 22.6% at 500 W and from 49.6 to 54.7% at 700 W, and changes nothing at 1000 W (81.2 versus 81.8%); truncating to 35 bits adds 1–2 points. The accuracy is then set by the truncation (5.1×10^{-7} at 19 bits), and dropping moduli is the better trade: 8 moduli on full operands (2.6×10^{-8}) are both faster (+76.6% at 500 W) and more accurate than 12 moduli on 19-bit operands. Within emulation, the number of moduli, not the operand mantissa, is the knob to turn.

IX. AI Workloads: Training and Inference (GH200)

MLPerf Training measures time to target quality (eval loss ≤ 0.925 on GovReport for Llama 2 70B LoRA); a faster step that needs more samples to converge loses.

TABLE XVI
WikiText-2 perplexity (sequence 2048) relative to BF16 for the linear layers' formats; FP8 uses real FP8 tensor-core GEMMs, and kept bits are rounded to nearest (ties to even).

Format	L2-7B	L3.1-8B	Qw2.5-7B	Mis-7B	OPT-6.7B	L2-70B
BF16, 4 b	+0.09	+0.19	+0.13	+0.09	+0.45	+0.10
BF16, 3 b	+0.44	+1.02	+0.65	+0.41	+2.00	+0.51
FP8 per tensor	+0.63	+0.95	+0.70	+0.45	+2.23	+0.48
FP8 per channel	+0.32	+0.88	+0.61	+0.34	+0.69	+0.42
FP8 p.c., 2 b	+3.2	+6.5	+4.0	+2.5	+7.0	+3.4
FP8, wts. 2 b	+1.9	+4.0	+2.6	+1.3	+2.8	—

Values in percent. b: kept bits; L: Llama, Qw: Qwen, Mis: Mistral; p.c.: per channel; wts.: weights only.

Training: truncating the FP8 base weights is a negative result. We truncated only the frozen FP8 base weights of Llama 2 70B LoRA (once, after loading); activations, gradients, LoRA weights and updates and per-tensor scaling follow [52]. On 8 nodes with the same seed, native FP8 at 575 W reached the target after 3456 samples in 28.34 min. Keeping 2 bits made each 384 samples only 0.9–1.2% faster but needed 4224 samples (22% slower at 575 W, 19% at 800 W); 1 bit did not converge (it also lowers the largest representable E4M3 value from 448 to 384, compressing the top weights by about 14%). The eval-loss curve shifts up by 0.006–0.008, while the train-loss gap varies (with 2 bits −0.031 at the first logged step, then +0.003 to +0.017): truncation weakens the base model at load time, and a rank-16 adapter does not compensate. A second seed (5678, 800 W) agrees: native 3072 samples in 22.74 min, 2 bits 4224 samples in 31.56 min (+39%). Across two seeds and four pairs (one at 575 W, three at 800 W), the first evaluation is always 0.008–0.010 higher and convergence takes 2–3 more evaluation intervals. Native convergence differs by one interval between node sets (3072 versus 3456), so we read this only as evidence that FP8's 3 mantissa bits leave no room. The rounding mode shows no consistent order (at 800 W truncation and stochastic rounding converge at 4224 samples, nearest at 5376). Truncating only MLP layers needs 4224 samples, attention only 3840 (one interval, within node-set noise, but attention is only about 18% of the linear weights); the first-evaluation gaps of +0.0064 (MLP) and +0.0028 (attention) roughly add up to the +0.0086 of full truncation (their sum is 0.0092). A repeat of the attention-only run gave nearly the same first evaluation (0.9428 versus 0.9425) but converged one interval later (4224 versus 3840, missing the target by 10^{-5} at 3840), so the first-evaluation gap, not the convergence point, is the robust metric.

Inference: FP8 has already taken most of the benefit. BF16 rounded to nearest at 3 bits has as many mantissa bits as FP8 E4M3 and matches its accuracy on six language models (Table XVI; with truncation, 4 bits are needed for a similar GEMM error, 4.35% on uniform data). We then compared the two where FP8 is known to fail: attention perplexity at 2k/8k context (BF16 3 bits +0.49/+1.06%, emulated FP8 attention +0.52/+1.08%), key retrieval at 8k–32k context on

Llama-3.1-8B (both 100%), BERT-large SQuAD (all formats above 99.9% of FP32 F1) and CogVideoX-2B video generation (PSNR against native 18–38 dB for FP8 attention, 20–29 dB for BF16 3 bits). With proper scaling the two tie, and FP8 GEMMs are about 1.7–1.9× as fast (at 575 W, 1.67–1.72× BF16 with 3 bits and 1.80–1.87× native BF16). A naive FP8 attention that does not scale the softmax probabilities fails as sequences grow (74% error at 4096 tokens), and on CogVideoX-2B (about 18k tokens) it changes the composition and adds noise (PSNR 10–12 dB, SSIM 0.07–0.66) where BF16 with 3 bits keeps 20–21 dB and SSIM 0.82–0.91. This is a range problem, not a mantissa problem: the smallest positive E4M3 value, about 2^{-9}, exceeds the mean attention probability over 18k tokens, while BF16 has 8 exponent bits; correct implementations already fix it inside FP8 [54].

Without switching to 2:4 structured sparsity or INT4 weights, zeroing on top of FP8 is the remaining way to go faster than FP8 on Hopper, which lacks FP4. On real layer-40 activations and weights of Llama2-70B (8192×57344×8192), keeping 2 bits on both sides is 4.4/3.2/5.0/8.2% faster at 1000/700/575/450 W, weights only 1.3–3.3%, but 70B perplexity rises by 3.4%. Per-layer sensitivity shows q/k/v as least sensitive but a small share of the compute (with grouped-query attention, q is about 7.8% and k and v about 1% each), and the MLP layers as most of the compute but more sensitive. FP8's advantage comes mainly from halving the data path: at 575 W BF16 with 3 bits is only 8% faster than BF16, while FP8 is 1.8×. Power savings are limited too. Decode (64 tokens) is bandwidth-bound; at 1000 W zeroing leaves its speed unchanged on uniform data, though on real Llama2-70B weights zeroed FP8 is still 2.6–3.0% faster at the same clock. FP8 zeroed to 0 bits saves 104 W on uniform data but only 61 W on real Llama2-70B weights, and truncating weights to 2 or 0 bits saves only 1.5% or 5.5% of power at an accuracy cost. Below 575 W these savings turn into 2–16% speedups, and prefill is power-bound even at 1000 W.

For weights and activations that directly set model quality and have no residual correction, zeroing turns directly into quality loss, in contrast with solvers such as HPL-MxP and SCF that correct errors. For GPUs without FP8 tensor cores, or pipelines that must stay in BF16, BF16 with 3–4 kept bits is a nearly lossless option; for FP8 deployments, zeroing saves only 1.5–5.5% of power in memory-bound phases, at an accuracy cost.

X. Discussion

Where truncation pays. Three conditions decide the gain: the GPU must be power-bound (or the saving shows only as lower power), the truncated operands must flow directly and in volume into multipliers or data paths, and the algorithm must tolerate the error, ideally by correcting it (refinement, self-consistency). The model of Section IV makes the first quantitative: the equivalent saving is a fixed fraction $1/R-1$ of the dynamic part of the cap, so a kernel's locked-clock energy ratio predicts its benefit under every cap. The second explains why GEMM-dominated solvers (HPL, HPL-MxP, DF-MP2, the GP likelihood) gain 4–20% while N-body, MFEM, SW4 and LAMMPS gain little. The third separates HPC solvers, which tolerate 20–36 bits, from AI weights, which have no residual correction (Section IX).

The stock limit is a power cap. On GH200 the gain at the default 1000 W module cap is mainly energy (7.5–8.6% less per HPL solve) with 0.6–3.9% more speed. On MI350X the default 1000 W package limit binds for FP64, BF16 and FP8 matrix work, for HPL, HPL-MxP, DF-MP2 and the GP likelihood alike, so truncation turns directly into 4.6–10% more speed at stock settings. As accelerators raise their throughput faster than their power envelope, running at the limit becomes the default operating point rather than a datacenter policy, and the energy per operation, including the share spent on toggling unneeded bits, becomes a first-order performance parameter.

Benefit under a fixed power budget. Under a fixed power budget, such as a rack limit, the equivalent power saving becomes usable compute. At 8 GH200 nodes × 700 W, tuned HPL with the safe 36 bits rises from 346.2 to 362.9 TFLOP/s on set A (+4.8%; set B +3.9%); native HPL would need about 768 W per node (774 W on set B), about 0.5–0.6 kW more for the cluster, to match. On one MI350X node at 800 W per GPU, the same 36 bits is worth 43–50 W per GPU, about 0.35–0.4 kW per node. The cost is one bitwise operation per element, which can be fused into the kernel that produces the operand, and no hardware. At 575 W the SM clocks of our 14 GH200 nodes differ by 17% (1129–1325 MHz), and synchronous runs move at the pace of the slowest node, as earlier work found for CPUs under power bounds [18], [19]; zeroing and variation-aware caps are complementary.

Truncation and emulation. INT8 emulation of FP64 is the stronger lever where its throughput advantage is large and it can amortize its fixed costs: on MI350X it is up to 2.2× native for large k and 1.7× in DF-MP2. Truncation remains the better choice where emulation cannot amortize (small k or n, many distinct shapes), where the cap is tight, where memory is full, where per-row exponent spans are wide, outside matrix multiplication, and for BF16 and FP8, for which no emulation exists. Applied inside emulation it adds only 1–4 points under a cap, and fewer moduli are the better way to trade accuracy for speed there (Section VIII-F). Replacing a BLAS call wholesale also carries an operational risk that truncation does not: the emulation library must be validated at the problem sizes actually run (Section VIII-E). A runtime that knows each call's shape, the cap and a cheap span estimate can choose per call.

Benchmark rules. The TOP500 Linpack rules require full 64-bit floating-point arithmetic and explicitly exclude solving in lower precision with iterative refinement, and emulating floating point with integer arithmetic [63]; HPL on truncated operands does not meet the full-precision requirement either. Since June 2025 the HPL-MxP rules require at least 16-bit floating point and allow diagonally dominant test matrices for testing only [60]; BF16 keeping 3 bits is effectively a 12-bit format, and our test matrix is not allowed for official runs.

All HPL and HPL-MxP numbers here are therefore research results, not official scores. MLPerf Inference closed division allows any mathematical quantization but forbids pruning [55]; zeroing is the former. Whether MLPerf Training permits truncated weights depends on each round's rules; in our results uniform truncation does not pay anyway. Scientific software has no such restriction; the criterion is whether physical results stay within the required accuracy.

A hardware counterexample. An Altera patent states that zeroing low bits at the input of an FPGA floating-point multiplier has "barely any effect on reducing the amount of power" because the compression tree keeps switching [64]. This does not contradict our measurements: the patent is a qualitative statement about a 27×27 fixed-point multiplier, whereas on a GPU the truncated bits also travel through registers, operand buses and memory, whose toggles disappear as well. Table I supports this: a pure memory copy saves 9–18% of power without any multiplication, mostly from the absence of toggles, while in the multipliers of both GPUs zeros still save more than ones (Section III).

Limitations and threats to validity. (i) Two platforms, one generation each. The mechanism, the model and the direction of the cap effect on the emulation crossover hold on both, but the size of the gain depends on each part's power management, its INT8-to-FP64 ratio and its cooling, and other generations may differ. The MI350X results come from two nodes of one system, each with 8 GPUs, and the HPL and HPL-MxP numbers are single-node (8-GPU) runs, not multi-node. (ii) The HPL residual is a deterministic but chaotic function of N, and the safe bit count is platform-dependent (28 bits passes at three of four sizes on GH200 and at none on MI350X), so operating points (36 bits for HPL, 3 for HPL-MxP) were validated at several sizes on each platform. (iii) HPL-MxP used a diagonally dominant test matrix (diagonal equal to the row's off-diagonal magnitude sum) with approximate diagonal factorization, which the rules allow only for testing [60]; its stability at 3 kept bits may be a property of this matrix. (iv) Each scientific code was tested on one or two input systems; forces, geometry optimization and larger systems are not covered, and on MI350X DF-MP2 uses our own contraction of PySCF's tensor rather than GPU4PySCF. (v) The GH200 shim costs 3–9% for copies depending on the application, so shim results are always compared with a baseline taking the same copy path; in-program zeroing has no such cost. (vi) GH200 nodes differ by 13–17% at equal power and the GPUs of one MI350X node by up to 30% in clock in synchronous HPL-MxP, so all comparisons are paired on the same nodes or GPUs by ABBA order or by seed; intervals treat pairs within a job or node as independent, and moving HPL to a second GH200 node set lowered its gain at 600–800 W by 0.5–0.9 points, more than pairs within a job differ, while the two MI350X nodes agree within 2 points. (vii) The default caps are also the highest, not uncapped states; the limiters' time scales are unknown; and GH200 caps below 450 W lie outside the datasheet range [56]. (viii) Cooling changes conclusions: on GH200 at low fan speed the GPU hits its thermal limit first and at full fan speed native HPL-MxP at 1000 W trips the hardware power brake; on MI350X, chassis fans reacting to the previous run biased one multi-variant comparison (Section VI). The GEMM sweeps, the GP and the DF-MP2 runs use multi-variant orders; they were replicated on a second node (GEMM) or run as ABBA within one process, which cancels linear drift but not this nonlinear effect. (ix) The GH200 cluster is shared and another user's tool reset caps at least twice, so every window checks the cap in effect. (x) Power comes from the devices' own sensors, not an external meter. On MI350X energy comes from a hardware accumulator rather than sampled power, but it is still the device's own estimate. GH200 module power also includes the Grace CPU and memory; all energy figures are relative values on one path per platform. (xi) The clock reduction we attribute to circuit-level clock stretching on MI350X is an inference: no available counter reports it. The model's residual errors grow where the truncated program runs at clocks the native program never reaches. (xii) Not all of the Quantum ESPRESSO gain is a power effect: on the gold surface about half comes from fewer Davidson iterations, for reasons not yet identified. (xiii) The real-data emulation comparisons on Quantum ESPRESSO ran once per setting, and GEMMul8 ran out of GPU memory on two GH200 applications and on MI350X HPL at the largest N. (xiv) The Gaussian-process results come from our ExaGeoStat-like kernel on synthetic data, and its mixed-precision variant is a simplified re-implementation; ExaGeoStat itself was measured only through the shim, whose copy cost offsets the gain.

XI. Conclusion

On production GPUs of two vendors, truncating unused mantissa bits in software buys real performance for real HPC applications under power limits while their physical results stay within each code's own criterion. The mechanism is the same on both: bits that stop toggling, in matrix engines, vector units and data paths, account for most of the saved energy, and a two-parameter model built on it predicts the gain under any cap from one calibration. What differs is the operating point: GH200 gains mainly under lowered caps, whereas MI350X is power-bound at its stock limit, so FP64 HPL keeping 36 bits is 4.6–5.1% faster, HPL-MxP keeping 3 bits on a diagonally dominant test matrix 4.9–5.1% (7.6 to 8.0 PFLOP/s per node), a Gaussian-process likelihood 7.6% and DF-MP2 10.2% faster at default settings, and 6–20% at 500 W. Microbenchmarks give upper bounds (up to 65% for FP64 with 0 kept bits at 400 W), not application operating points. The scope is equally clear. INT8 emulation of FP64 is faster for long inner dimensions at generous caps, by up to 2.2× on MI350X, but a tighter cap moves the crossover toward truncation on both platforms, emulation needs workspace that limits problem size and warm-up that penalizes many distinct shapes, and on data with a wide per-row exponent span fixed-width emulation can silently go wrong; replacing every GEMM with FP32 or 3xTF32 fails on long inner products; kernels whose intermediates regrow full mantissas gain little;

TABLE XVII
WHERE EACH MECHANISM AND EXPERIMENT LIVES IN THE REPOSITORY.

Path	Content
gh200/bench/, mi350/bench/	mantissa GEMM microbenchmarks (cuBLAS; rocBLAS/hipBLASLt with GEMMul8 hook, transposes, exponent spread) and non-GEMM FP64 kernels
gh200/shim/mantshim.cu	cuBLAS interposer: rounding modes, cuBLAS emulation (fixed and automatic, bit-count log), FP32/3xTF32 replacement, exponent-span statistics
rocHPL/src/blas/HPL_quantize_device.cpp	HPL operand rounding (HPL_QUANT_*_BITS); the AMD build is branch feat/adaptive-mixed-storage
rocHPL-MxP/src_mi350/hplmxp.h	HPL-MxP zeroing on BF16 write-back (compile-time MANT_BITS), same source for both platforms
apps/q-e/KS_Solvers/Davidson/	in-program truncation and the tolerance-driven rule (QE_MANT_ADAPT)
apps/pyscf-inputs/mp2_variants.py, mi350/apps/mp2_prep.py, mi350/apps/dfmp2.hip	DF-MP2 variants on GH200; tensor preparation and HIP contraction on MI350X
apps/geostat/gp_mle.py, mi350/apps/gp_chol.hip	Gaussian-process likelihood and MLE (CUDA/cupy and HIP)
apps/patches/	MFEM, SW4, LAMMPS and ExaGeoStatCPP changes
gh200/slurm/, mi350/scripts/, mi350/tools/	job scripts (cap sweeps with ABBA or pairwise ABBA order); MI350X power-cap tool and amdsmi sampler with energy accumulator
gh200/analysis/, mi350/analysis/	summaries, ΔW, intervals, crossover fit, power model (power_model.py), figures
gh200/results/, mi350/results/	summarized CSV files behind every table and figure

and for FP8 AI inference, further zeroing trades accuracy for speed. Safe bit counts must be validated on the target platform, and measurements must control for cooling feedback. Mantissa zeroing costs one bitwise operation per element, needs no hardware support, and complements format changes. All HPL, HPL-MxP and MLPerf numbers here are research results, not official scores.

DATA AND CODE AVAILABILITY

The mantissa shim, the in-program truncation of each application, the job scripts, the raw measurement logs, the summarized CSV files and the scripts that draw every figure are in the project repository (branch mi350-study). gh200/analysis/paper_figures.py and mi350/analysis/paper_figures_mi350.py regenerate every figure of this paper from the CSV files in gh200/results/ and mi350/results/.

APPENDIX A
ARTIFACT DESCRIPTION

Hardware and power control. GH200: fourteen nodes, each with one NVIDIA GH200 480GB superchip (aarch64), 200 Gb/s InfiniBand, Slurm partition compute; module power caps are set with the cluster's nv_power tool only on exclusively allocated nodes; fans were left at the operator's setting. MI350X: two nodes with 8 MI350X OAM GPUs each, Slurm partition gpu; package power limits (PPT0) are set per GPU with amd-smiset-oPPT0 and read back from sysfs; clocks for the locked-clock experiments with amd-smiset-Lsclkmax. Every job restores the default limit (and clock) on exit and records the cap in effect in each measurement window.

Software. GH200: NVHPC 26.5 (CUDA 13.2, cuBLAS 13.4, NCCL 2.29, HPC-X Open MPI 5), driver 615.71; ExaGeoStatCPP links cuBLAS 13.6. HPL is a CUDA port of rocHPL (branch feat/cuda-port) and HPL-MxP the perf/h200-mantissa branch of an HPL-MxP implementation [33]; Quantum ESPRESSO 7.6 [57]; GPU4PySCF 1.8.1 [58], [59]; MFEM 4.10.1 (CUDA, partial assembly); SW4 (raja branch, commit 52b51c9, RAJA 2026.07.1); LAMMPS 22 Jul 2025 Update 6 (KOKKOS, CUDA); GEMMul8 3.5.1; MLPerf Training Llama 2 70B LoRA with NVIDIA's v5.0 implementation. MI350X: ROCm 7.2.3 (rocBLAS, hipBLASLt, rocSOLVER, RCCL), amdgpu 6.16.13, Open MPI 5.0.7 with UCX; rocHPL branch feat/adaptive-mixed-storage (commit 37d0475), the same HPL-MxP source; GEMMul8 3.2.0 (commit 833e576); PySCF 2.14. Analysis uses Python with pandas, NumPy and Matplotlib.

Code and data. Table XVII lists where each part lives; the experiment logs in gh200/ and mi350/ record every job, including failed, cancelled and repeated ones, and every table and figure names its jobs and result files there.

Reproduction. Each experiment is one job that sets caps only on its own nodes, waits for the GPUs to cool before each run, interleaves variants in ABBA (or pairwise ABBA) order and writes power logs next to the program output; an analysis script then joins run windows with the power logs. For example, the GH200 crossover sweep of Section VIII-B is gh200/slurm/e1_gemm_tdp.sbatch with the shapes, caps and methods passed as environment variables, summarized by gh200/analysis/crossover.py; the MI350X sweeps are mi350/scripts/e1_gemm_tdp.sh (GEMM, crossover, locked clocks via e1b_fixedclk.sh), hpl_tdp.sh (HPL and HPL-MxP), gp_tdp.sh and mp2_tdp.sh, submitted through mi350/slurm/run.sbatch and summarized by mi350/analysis/e1_summary.py, hpl_summary.py, gp_summary.py and mp2_summary.py; the model is mi350/analysis/power_model.py.

References

[1] S. Bhalachandra, B. Austin, S. Williams, and N. J. Wright, "Understanding the impact of input entropy on FPU, CPU, and GPU power," arXiv preprint arXiv:2212.08805, 2022.

[2] T. Gregersen, P. Patel, and E. Choukse, "Input-dependent power usage in GPUs," in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1872–1877.

[3] H. He, "Strangely, matrix multiplications on GPUs run faster when given "predictable" data! [short]," Thonk From First Principles (Substack blog), https://www.thonking.ai/p/strangely-matrix-multiplications, Apr. 2024.

[4] O. Antepara *et al.*, "Benchmark-driven models for energy analysis and attribution of GPU-accelerated supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, 2025, pp. 888–904.

[5] NVIDIA, "Performance benchmarking — NVIDIA TensorRT (GPU power consumption and power throttling)," https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/benchmarking.html, accessed Sep. 30, 2026.

[6] J. Y. F. Tong, D. Nagle, and R. A. Rutenbar, "Reducing power by optimizing the necessary precision/range of floating-point arithmetic," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 8, no. 3, pp. 273–286, 2000.

[7] J. Pool, A. Lastra, and M. Singh, "Energy-precision tradeoffs in mobile graphics processing units," in *IEEE International Conference on Computer Design (ICCD)*, 2008, pp. 60–67.

[8] ——, "Precision selection for energy-efficient pixel shaders," in *ACM SIGGRAPH Symposium on High Performance Graphics (HPG)*, 2011, pp. 159–168.

[9] D. Guenther, T. Henriksson, R. Leupers, and G. Ascheid, "Mantissa-masking for energy-efficient floating-point LTE uplink MIMO baseband processing," in *Design, Automation & Test in Europe (DATE)*, 2016, pp. 1028–1029.

[10] P. V. Argade and D. N. Peroni, "Variable precision computing system," US Patent 11,455,766 B2, Sep. 27, 2022.

[11] K. Ozaki, Y. Uchino, and T. Imamura, "Ozaki scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique," arXiv preprint arXiv:2504.08009, 2025.

[12] Y. Uchino *et al.*, "High-performance and power-efficient emulation of matrix multiplication using INT8 matrix engines," in *SC '25 Workshops*, 2025.

[13] N. Chatterjee *et al.*, "Architecting an energy-efficient DRAM system for GPUs," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 73–84.

[14] S. Ghose *et al.*, "What your DRAM power models are not telling you: Lessons from a detailed experimental study," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, 2018.

[15] O. Antepara, L. Oliker, and S. Williams, "Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200," in *SC '25 Workshops*, 2025, pp. 1205–1216.

[16] A. Ujeniya, J. Eitzinger, G. Hager, and G. Wellein, "Architectural tradeoffs in the energy-efficient era: A comparative study of power-capping NVIDIA H100 and H200," arXiv preprint arXiv:2604.11391, 2026.

[17] J. Lucas and B. Juurlink, "ALUPower: Data dependent power consumption in GPUs," in *IEEE 24th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2016, pp. 95–104.

[18] B. Rountree, D. H. Ahn, B. R. de Supinski, D. K. Lowenthal, and M. Schulz, "Beyond DVFS: A first look at performance under a hardware-enforced power bound," in *IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum (IPDPSW)*. IEEE, 2012, pp. 947–953.

[19] Y. Inadomi *et al.*, "Analyzing and mitigating the impact of manufacturing variability in power-constrained supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015, pp. 1–12, article 78.

[20] T. Patki, D. K. Lowenthal, B. Rountree, M. Schulz, and B. R. de Supinski, "Exploring hardware overprovisioning in power-constrained, high performance computing," in *Proceedings of the 27th International ACM Conference on Supercomputing (ICS '13)*. ACM, 2013, pp. 173–182.

[21] J. You, J.-W. Chung, and M. Chowdhury, "Zeus: Understanding and optimizing GPU energy consumption of DNN training," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 119–139. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/you

[22] A. Angerd, E. Sintorn, and P. Stenström, "A framework for automated and controlled floating-point accuracy reduction in graphics applications on GPUs," *ACM Transactions on Architecture and Code Optimization*, vol. 14, no. 4, 2017, article 46.

[23] R. Jahadi and E. Atoofian, "Low-power register file for tensor cores," in *IEEE 15th International Green and Sustainable Computing Conference (IGSC)*, 2024, pp. 159–164.

[24] A. Dawson and P. D. Düben, "rpe v5: An emulator for reduced floating-point precision in large numerical simulations," *Geoscientific Model Development*, vol. 10, pp. 2221–2230, 2017.

[25] N. J. Higham and S. Pranesh, "Simulating low precision floating-point arithmetic," *SIAM Journal on Scientific Computing*, vol. 41, no. 5, pp. C585–C602, 2019.

[26] M. O'Connor *et al.*, "Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.

[27] G. Pekhimenko, E. Bolotin, N. Vijaykumar, O. Mutlu, T. C. Mowry, and S. W. Keckler, "A case for toggle-aware compression for GPU systems," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2016, pp. 188–200.

[28] C. Rubio-González *et al.*, "Precimonious: Tuning assistant for floating-point precision," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013.

[29] N. J. Higham and T. Mary, "Mixed precision algorithms in numerical linear algebra," *Acta Numerica*, vol. 31, pp. 347–414, 2022.

[30] A. C. I. Malossi *et al.*, "The transprecision computing paradigm: Concept, design, and applications," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 1105–1110.

[31] T. Grützmacher, T. Cojean, G. Flegar, H. Anzt, and E. S. Quintana-Ortí, "Acceleration of PageRank with customized precision based on mantissa segmentation," *ACM Transactions on Parallel Computing*, vol. 7, no. 1, pp. 1–19, 2020.

[32] A. Haidar, S. Tomov, J. Dongarra, and N. J. Higham, "Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC18)*, 2018, pp. 603–613.

[33] J. Dongarra and P. Luszczek, "HPL-MxP benchmark: Mixed-precision algorithms, iterative refinement, and scalable data generation," *The International Journal of High Performance Computing Applications*, vol. 40, no. 1, pp. 52–62, 2026, published online Sep. 25, 2025; arXiv:2509.19618.

[34] M. Klöwer, M. Razinger, J. J. Dominguez, P. D. Düben, and T. N. Palmer, "Compressing atmospheric data into its real information content," *Nature Computational Science*, vol. 1, no. 11, pp. 713–724, 2021.

[35] M. Patrou *et al.*, "Power-capping metric evaluation for improving energy efficiency in HPC applications," in *ISC High Performance 2025 Workshops (EESP)*, ser. LNCS, 2025, pp. 231–244.

[36] Z. Yang, K. Adamek, and W. Armour, "Accurate and convenient energy measurements for GPUs: A detailed study of NVIDIA GPU's built-in power sensor," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC24)*, 2024.

[37] K. Ozaki, T. Ogita, S. Oishi, and S. M. Rump, "Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications," *Numerical Algorithms*, vol. 59, no. 1, pp. 95–118, 2012.

[38] H. Ootomo, K. Ozaki, and R. Yokota, "DGEMM on integer matrix multiplication unit," *The International Journal of High Performance Computing Applications*, vol. 38, no. 4, pp. 297–313, 2024.

[39] NVIDIA, "cuBLAS library documentation, section 1.5: Floating point emulation (v13.4; archived 13.2.1)," https://docs.nvidia.com/cuda/cublas/index.html#floating-point-emulation, accessed Oct. 5, 2026.

[40] A. Schwarz *et al.*, "Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme," in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia)*, 2026, pp. 91–101.

[41] S. Abdulah, H. Ltaief, Y. Sun, M. G. Genton, and D. E. Keyes, "ExaGeoStat: A high performance unified software for geostatistics on manycore systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 12, pp. 2771–2784, 2018.

[42] ——, "Geostatistical modeling and prediction using mixed precision tile Cholesky factorization," in *IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2019, pp. 152–162.

[43] S. Abdulah *et al.*, "Accelerating geostatistical modeling and prediction with mixed-precision computations: A high-productivity approach with PaRSEC," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 964–976, 2022.

[44] Q. Cao *et al.*, "Reshaping geostatistical modeling and prediction for extreme-scale environmental applications," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022, pp. 1–12.

[45] J. Ren, H. Ltaief, S. Abdulah, and D. E. Keyes, "Accelerating mixed-precision out-of-core Cholesky factorization with static task scheduling," in *ISC High Performance 2025 Research Paper Proceedings (40th International Conference)*, 2025, pp. 1–12.

[46] S. Abdulah *et al.*, "Boosting earth system model outputs and saving PetaBytes in their storage using exascale climate emulators," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024, pp. 1–12.

[47] Q. Cao, S. Abdulah, H. Ltaief, M. G. Genton, D. Keyes, and G. Bosilca, "Reducing data motion and energy consumption of geospatial modeling applications using automated precision conversion," in *IEEE International Conference on Cluster Computing (CLUSTER)*, 2023, pp. 330–342.

[48] R. Olivares-Amaya, M. A. Watson, R. G. Edgar, L. Vogt, Y. Shao, and A. Aspuru-Guzik, "Accelerating correlated quantum chemistry calculations using graphical processing units and a mixed precision matrix multiplication library," *Journal of Chemical Theory and Computation*, vol. 6, no. 1, pp. 135–144, 2010.

[49] J. Woo, S. Kim, and W. Y. Kim, "Dynamic precision approach for accelerating large-scale eigenvalue solvers in electronic structure calculations on graphics processing units," *Journal of Chemical Theory and Computation*, vol. 19, no. 5, pp. 1457–1465, 2023.

[50] H. Huang, W. Shao, and J. Hammond, "Accelerating density fitting with adaptive-precision and 8-bit integer on AI accelerators," arXiv preprint arXiv:2601.08077, 2026.

[51] W. Dawson, K. Ozaki, J. Domke, and T. Nakajima, "Reducing numerical precision requirements in quantum chemistry calculations," *Journal of Chemical Theory and Computation*, vol. 20, no. 24, pp. 10 826–10 837, 2024.

[52] P. Micikevicius, D. Stosic *et al.*, "FP8 formats for deep learning," arXiv preprint arXiv:2209.05433, 2022.

[53] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 37, 2015, pp. 1737–1746.

[54] J. Zhang, H. Huang, P. Zhang, J. Wei, J. Zhu, and J. Chen, "SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, arXiv:2411.10958.

[55] MLCommons, "MLPerf inference rules, section 8.1: Weight definition and quantization," https://github.com/mlcommons/inference_policies/blob/master/inference_rules.adoc, accessed Oct. 5, 2026.

[56] NVIDIA, "NVIDIA GH200 Grace Hopper superchip datasheet," Document 3773000, linked from https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/, Mar. 2025, accessed Oct. 5, 2026.

[57] P. Giannozzi *et al.*, "Quantum ESPRESSO toward the exascale," *The Journal of Chemical Physics*, vol. 152, p. 154105, 2020.

[58] X. Wu *et al.*, "Enhancing GPU-acceleration in the Python-based simulations of chemistry frameworks," *WIREs Computational Molecular Science*, vol. 15, no. 2, p. e70008, 2025.

[59] R. Li, Q. Sun, X. Zhang, and G. K.-L. Chan, "Introducing GPU acceleration into the Python-based simulations of chemistry framework," *The Journal of Physical Chemistry A*, vol. 129, no. 5, pp. 1459–1468, 2025.

[60] HPL-MxP, "HPL-MxP rules," https://hpl-mxp.org/rules.md, accessed Oct. 5, 2026.

[61] M. Fasi, N. J. Higham, M. Mikaitis, and S. Pranesh, "Numerical behavior of NVIDIA tensor cores," *PeerJ Computer Science*, vol. 7, p. e330, 2021.

[62] F. A. Khattak and M. Mikaitis, "Accurate models of NVIDIA tensor cores," arXiv preprint arXiv:2512.07004, 2025.

[63] TOP500.org, "The Linpack benchmark," https://www.top500.org/project/linpack/, accessed Oct. 5, 2026.

[64] M. Langhammer, "Low power optimizations for a floating point multiplier," US Patent 9,606,608 B1 (Altera Corporation), 2017.